VIGNAN'S INSTITUTE OF INFORMATION TECHNOLOGY (AUTONOMOUS)
DUVVADA - VISAKHAPATNAM – 530049
(An Autonomous Institute, accredited by NAAC, Affiliated to JNTUGV, AP)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(DATA SCIENCE)

MEAN STACK TECHNOLOGIES LAB MANUAL

(Course Code: 1044233180)

VR23

III Year-I Semester



Prepared By

Mr.B.V.RAMANA

Assistant Professor(Sr. Grade)

VIGNAN'S INSTITUTE OF INFORMATION TECHNOLOGY

(Autonomous)

Affiliated to JNTUGV, Vizianagaram & Approved by AICTE, NewDelhi Re-Accredited by NAAC (CGPA of 3.41/ 4.00)

ISO 9001:2008,ISO 14001:2004,OHSAS 18001:2007 Certified Institution

VISAKHAPATNAM–530039

VISION

We envision to be a recognized leader in technical education and shall aim at national excellence by creating competent & socially conscious technical manpower for the current and the future industrial requirements and development of the nation.

MISSION

- Introducing innovative practices of teaching & learning.
- Undertaking Research & Development in thrust areas.
- Continuously collaborating with industry.
- Promoting a strong set of ethical values.
- Serving the surrounding region and the nation at large.

VISION OF THE DEPARTMENT

To be a pioneer in Data Science education, nurturing graduates capable of deriving actionable insights and contributing to data-driven innovations for societal and industrial growth.

MISSION OF THE DEPARTMENT

- To provide comprehensive knowledge in data analytics, machine learning, and big data technologies.
- To encourage innovation and interdisciplinary research for solving real-world problems.
- To nurture industry-ready professionals with ethical values and societal commitment.

COURSE OBJECTIVES:

- To provide hands-on knowledge of developing full-stack web applications using the MEAN (MongoDB, Express.js, Angular, Node.js) stack.
- To enable students to design responsive front-end interfaces using HTML5, CSS, JavaScript, and Angular.
- To develop skills in building scalable backend services and RESTful APIs using Node.js and Express.js.
- To understand database connectivity and operations using MongoDB for data storage and retrieval.

COURSE OUTCOMES:

CO	Course Outcome Statement	Bloom's Cognitive Level
CO1	Develop dynamic and interactive web pages using HTML5 and JavaScript, incorporating DOM manipulation and event handling.	L3
CO2	Build and secure RESTful web services using Node.js and Express.js for backend development.	L3
CO3	Perform CRUD operations and manage data efficiently using MongoDB with TypeScript-based implementation.	L4
CO4	Design and implement responsive single-page applications using Angular with routing, services, and dependency injection.	L5

LIST OF EXPERIMENTS:

UNIT I: HTML5 & JavaScript Essentials HTML5:

Structure, Forms, Tables, Media, Security Practices. JavaScript: Syntax, Functions, DOM Manipulation, Async Programming.

Tasks:

1. Create a static web page using HTML5 elements (headings, images, lists, forms).
2. Add validation logic to the HTML form using JavaScript functions.
3. Build an interactive component with DOM manipulation and event handling.
4. Fetch data from an API using JavaScript Fetch API and display it on the page.

UNIT II: Node.js & Express.js

Node.js: Core modules, NPM, file system, creating a server.Express.js: Routing, Middleware, API development, Security practices.

Tasks:

1. Create a simple Node.js web server that returns a response to browser requests.
2. Use Express.js to define RESTful routes and return JSON data.
3. Connect Express app to MongoDB and perform basic CRUD operations.
4. Implement session management using cookies and Helmet middleware.

UNIT III: MongoDB & TypeScript

MongoDB: CRUD, indexing, aggregation, collections. TypeScript: Interfaces, Classes, Generics, Modules.

Tasks:

5. Install MongoDB and create a cluster using MongoDB Atlas.
6. Write MongoDB CRUD queries to insert, update, delete documents.
7. Create a TypeScript app using interfaces, classes, and functions,
8. Build a module-based project in TypeScript connecting to a MongoDB collection.

UNIT IV: Angular Basics & Component Development

Angular setup, Components, Templates, Data Binding.Directives (@if, @for, @switch), Pipes, Forms.

1. Create a new Angular component and display content using interpolation.
2. Implement @if and @for to dynamically show/hide elements and list data.
3. Build a login form with validation using a template-driven approach.
4. Use built-in pipes to transform data (uppercase, lowercase, date).

UNIT V: Angular Advanced Features & Backend Integration

Services, Dependency Injection, HttpClient, RxJS. Routing, Route Guards, Lazy Loading, Nested Routes,

Tasks:

1. Create an Angular service to fetch and display data using HttpClient.
2. Implement routing between components with RouterLink and route parameters.
3. Add a route guard to restrict access based on login status.
4. Enable lazy loading for a module and observe performance improvements.

UNIT I

HTML5 & JavaScript Essentials HTML5:

Structure, Forms, Tables, Media, Security Practices. JavaScript: Syntax, Functions, DOM Manipulation, Async Programming.

Tasks:

1. Create a static web page using HTML5 elements (headings, images, lists, forms).
2. Add validation logic to the HTML form using JavaScript functions.
3. Build an interactive component with DOM manipulation and event handling.
4. Fetch data from an API using JavaScript Fetch API and display it on the page.

Task 1: Create a Static Web Page Using HTML5 Elements

Aim:

To design a static web page using basic HTML5 elements such as headings, images, lists, and forms.

Requirements:

- Browser (Chrome/Firefox/Edge)
- Text editor (VS Code/Notepad++)

Procedure:

1. Create a new `.html` file.
2. Define structure with `<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`.
3. Add headings (`<h1>`–`<h6>`), paragraphs (`<p>`), and images (`<img>`).
4. Create ordered/unordered lists (`<ol>`, `<ul>`).
5. Design a simple form using `<form>`, `<input>`, `<label>`, `<button>`.
6. Save and open in a browser.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First HTML5 Page</title>
</head>
<body>
  <h1>Welcome to HTML5</h1>
  <p>This is a sample web page.</p>

  <img src="sample.jpg" alt="Sample Image" width="200">

  <h2>Topics</h2>
  <ul>
    <li>HTML5 Basics</li>
```

```
    <li>Forms</li>
    <li>Tables</li>
  </ul>

  <h2>Contact Form</h2>
  <form>
    <label>Name:</label>
    <input type="text" name="name" required><br>
    <label>Email:</label>
    <input type="email" name="email" required><br>
    <button type="submit">Submit</button>
  </form>
</body>
```

Output:

A static HTML5 page is created and displayed in the browser.

Welcome to HTML5

This is a sample web page.



Topics

- HTML5 Basics
- Forms
- Tables

Contact Form

Name:
Email:
Submit

Task 2: Add Validation Logic Using JavaScript Functions

Aim:

To validate form inputs using JavaScript functions.

Procedure

1. Extend Experiment 1's form.
2. Create a JavaScript function to validate required fields.
3. Use `onsubmit` or `addEventListener` for form validation.

4. Display alert/error messages if inputs are invalid.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First HTML5 Page</title>
</head>
<body>
 <form id="regForm">
  <label>Name:</label>
  <input type="text" id="name"><br>
  <label>Email:</label>
  <input type="email" id="email"><br>
  <button type="submit">Register</button>
</form>

<script>
document.getElementById("regForm").onsubmit = function(e) {
  let name = document.getElementById("name").value;
  let email = document.getElementById("email").value;

  if(name === "" || email === "") {
    alert("All fields are required!");
    e.preventDefault();
  }
  alert(`${name}, ${email}`);
};
</script>
</body>
</html>
```

Output:

The form validates input fields using JavaScript before submission.



Task 3: Interactive Component with DOM Manipulation

Aim

To build an interactive component by manipulating DOM elements and handling events.

Procedure

1. Create an HTML button and container.
2. Write JavaScript to change content dynamically.
3. Add event listener (onclick, addEventListener).

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Manipulation</title>
</head>
<body>
<h2>Click Counter</h2>
<p id="count">0</p>
<button id="btn">Click Me</button>
<script>
let count = 0;
document.getElementById("btn").addEventListener("click", function() {
  count++;
  document.getElementById("count").textContent = count;
});
</script>
</body>
</html>
```

Output:

Clicking the button updates the counter dynamically.



Task 4: Fetch Data Using Fetch API

Aim

To fetch JSON data from an API using JavaScript Fetch API and display it on the page.

Procedure:

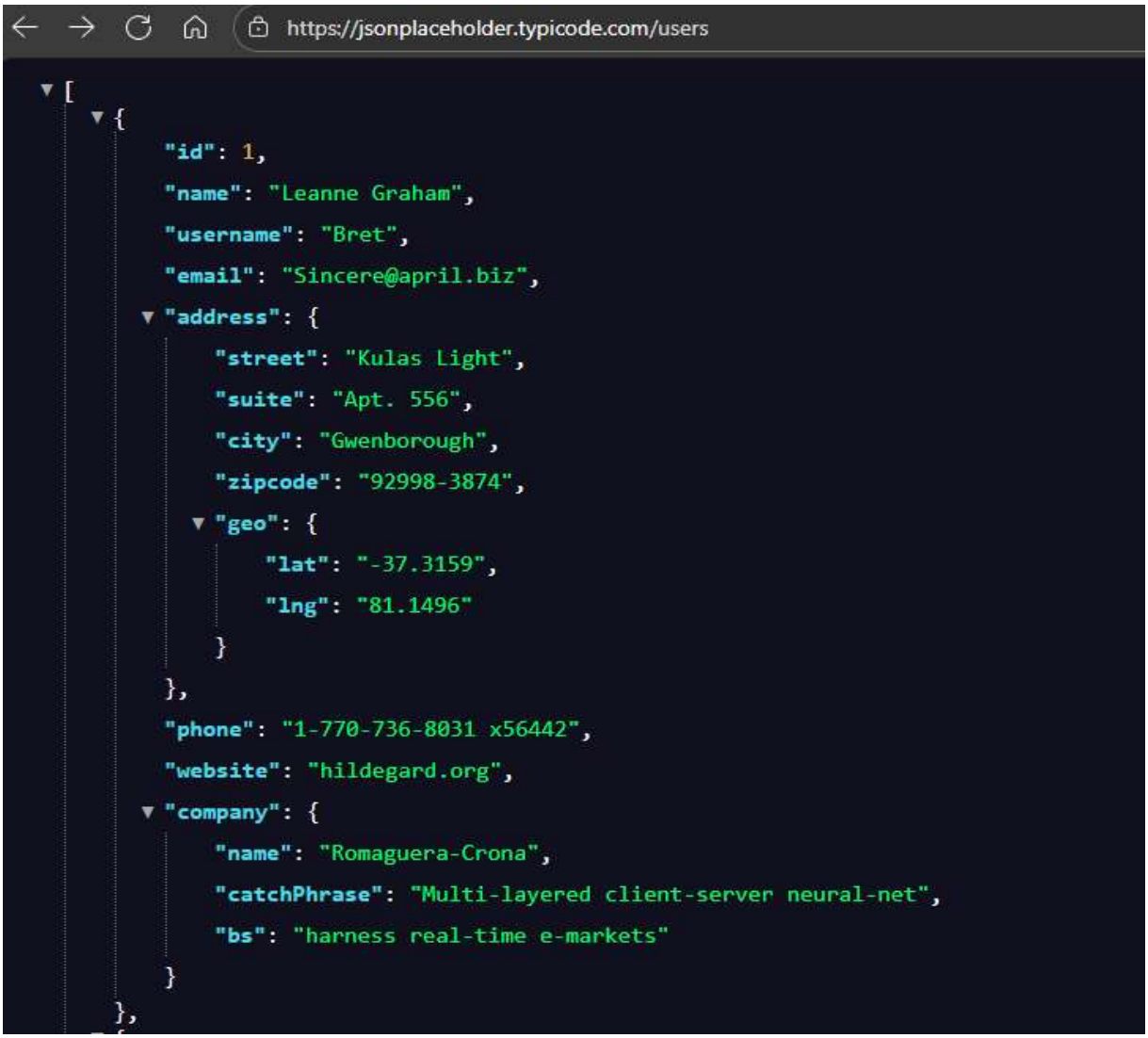
1. Use a public API (e.g., https://jsonplaceholder.typicode.com/users).
2. Write JavaScript fetch() to retrieve data.
3. Convert response to JSON.
4. Display Outputs in the DOM.

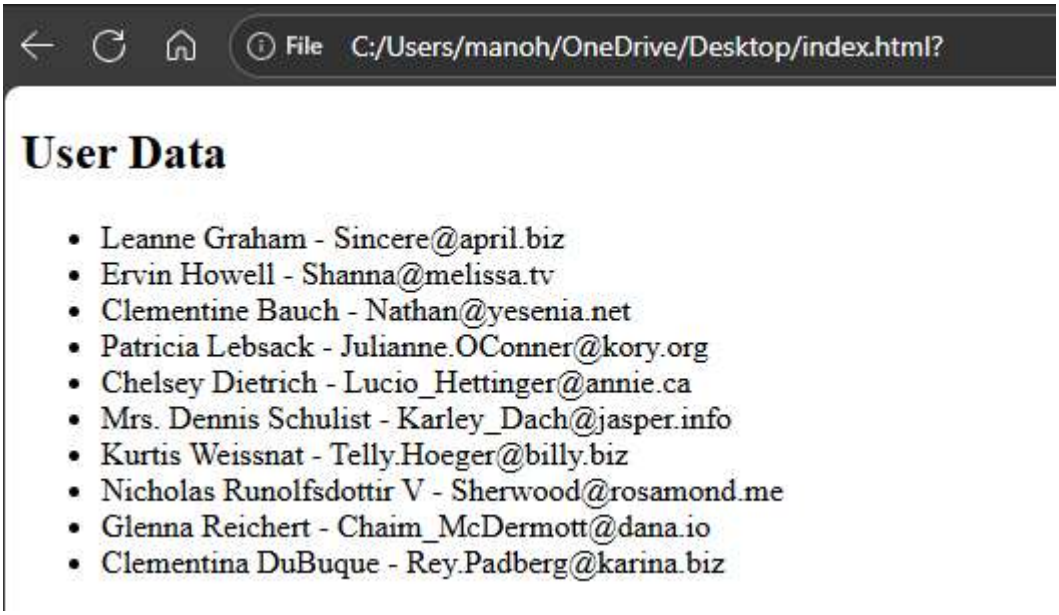
Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Fetch Data Using Fetch API</title>
</head>
<body>
<h2>User Data</h2>
<div id="users"></div>
<script>
fetch("https://jsonplaceholder.typicode.com/users")
.then(response => response.json())
.then(data => {
let output = "<ul>";
data.forEach(user => {
output += `<li>${user.name} - ${user.email}</li>`;
});
output += "</ul>";
document.getElementById("users").innerHTML = output;
})
.catch(error => console.error("Error:", error));
</script>
</body>
</html>
```

Output:

User data is fetched from the API and displayed on the web page.





UNIT II: Node.js & Express.js

Node.js: Core modules, NPM, file system, creating a server.Express.js: Routing, Middleware, API development, Security practices.

Tasks:

1. Create a simple Node.js web server that returns a response to browser requests.
2. Use Express.js to define RESTful routes and return JSON data.
3. Connect Express app to MongoDB and perform basic CRUD operations.
4. Implement session management using cookies and Helmet middleware.

Task 1: Create a Simple Node.js Web Server

Aim:

To create a basic Node.js server that responds to browser requests.

Requirements:

- Node.js installed
- Text editor (VS Code)

Procedure:

- Initialize project: `npm init -y`.
- Create `server.js`.
- Use the `http` core module to create a server.
- Send response on request.
- Run with `node server.js`.

Code:

```
@Maharshi → ~\OneDrive\Desktop → npm init -y
Wrote to C:\Users\manoh\OneDrive\Desktop\package.json:

{
  "name": "desktop",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

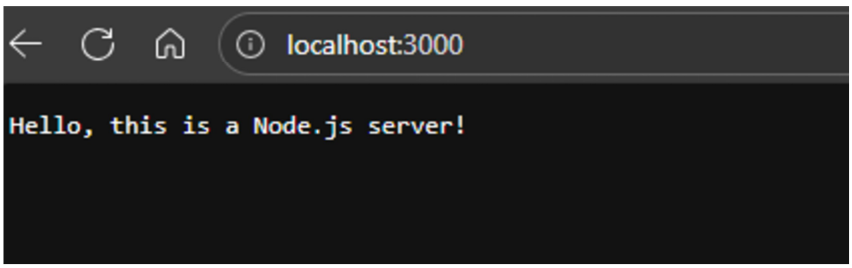
```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello, this is a Node.js server!');
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

Output:

A simple Node.js server responds with text in the browser.



Task 2: RESTful Routes with Express.js

Aim:

To define RESTful routes in Express.js and return JSON responses.

Procedure:

- Install Express: npm install express.
- Create app.js.
- RESTful Routes:
  - **GET** /api/users: Returns the list of all users.
  - **GET** /api/users/:id: Returns a specific user by ID or a 404 error if not found.
  - **POST** /api/users: Creates a new user with a name provided in the request body.
  - **PUT** /api/users/:id: Updates the name of an existing user by ID.
  - **DELETE** /api/users/:id: Deletes a user by ID.
- Return JSON responses.
- Start server with node app.js.

Code:

```
@Maharshi →node(22.17.1) ~\OneDrive\Desktop\lab-meanstak →npm install express

added 67 packages, and audited 68 packages in 11s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

In app.js:

```
const express = require('express');
const app = express();
app.use(express.json());

let users = [
     { id: 1, name: 'Alice', email: 'alice@example.com' },
     { id: 2, name: 'Bob', email: 'bob@example.com' },
];

// GET all users
app.get('/api/users', (req, res) => {
     res.json(users);
});

// GET single user
app.get('/api/users/:id', (req, res) => {
     const user = users.find((u) => u.id === parseInt(req.params.id));
     if (!user) return res.status(404).json({ message: 'User not found' });
     res.json(user);
});

// POST new user
app.post('/api/users', (req, res) => {
     const newUser = {
          id: users.length + 1,
          name: req.body.name,
          email: req.body.email,
     };
     users.push(newUser);
     res.json({ message: 'User added', data: newUser });
});

// PUT update user
app.put('/api/users/:id', (req, res) => {
     const user = users.find((u) => u.id === parseInt(req.params.id));
     if (!user) return res.status(404).json({ message: 'User not found' });

     user.name = req.body.name || user.name;
     user.email = req.body.email || user.email;

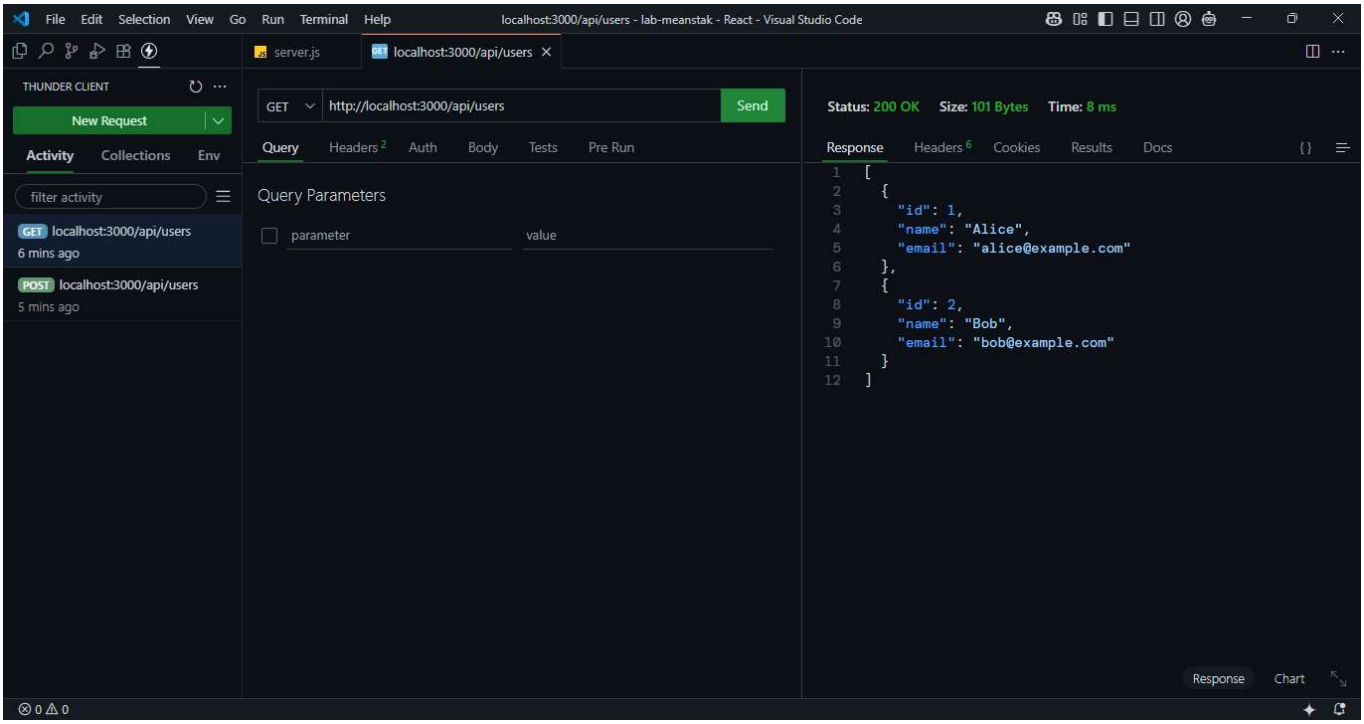
     res.json({ message: 'User updated', data: user });
});

// DELETE user
app.delete('/api/users/:id', (req, res) => {
```

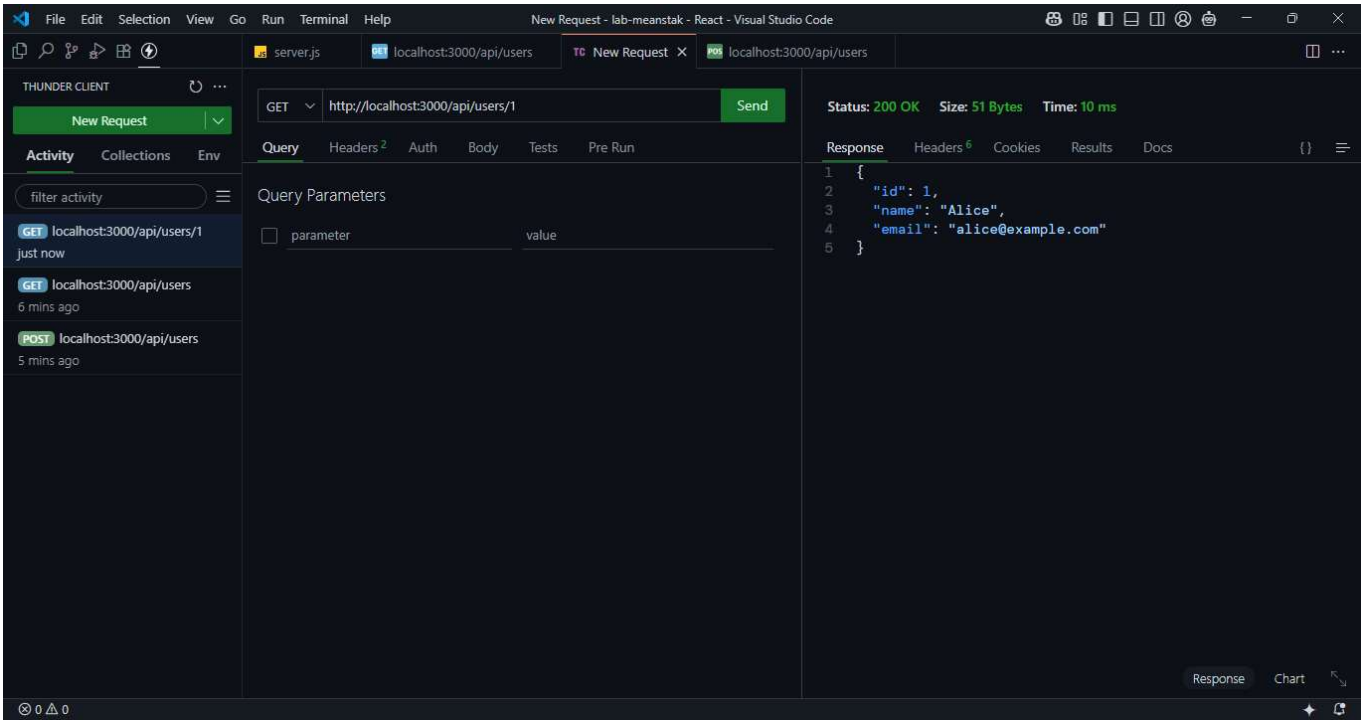
```
@Maharshi →node(22.17.1) ~\OneDrive\Desktop\lab-meanstak →node server.js
Express running on port 3000
```

Output:

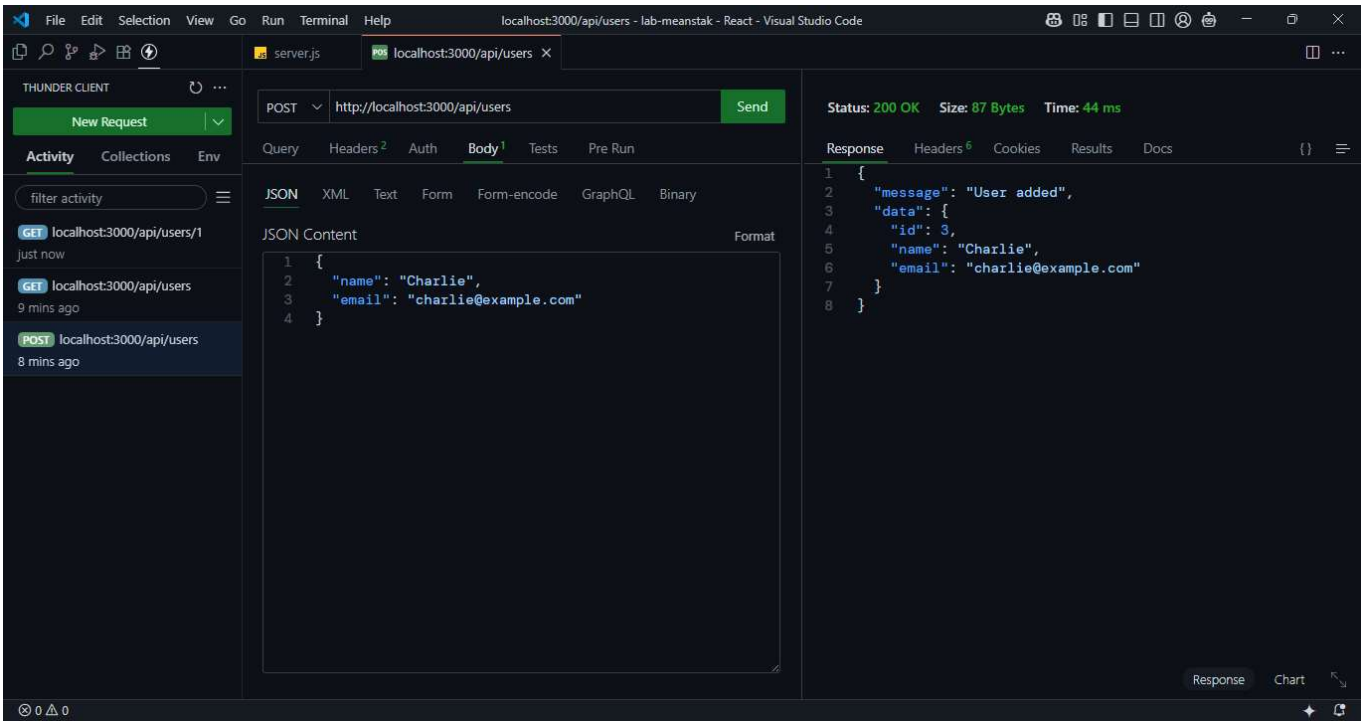
For GET request:



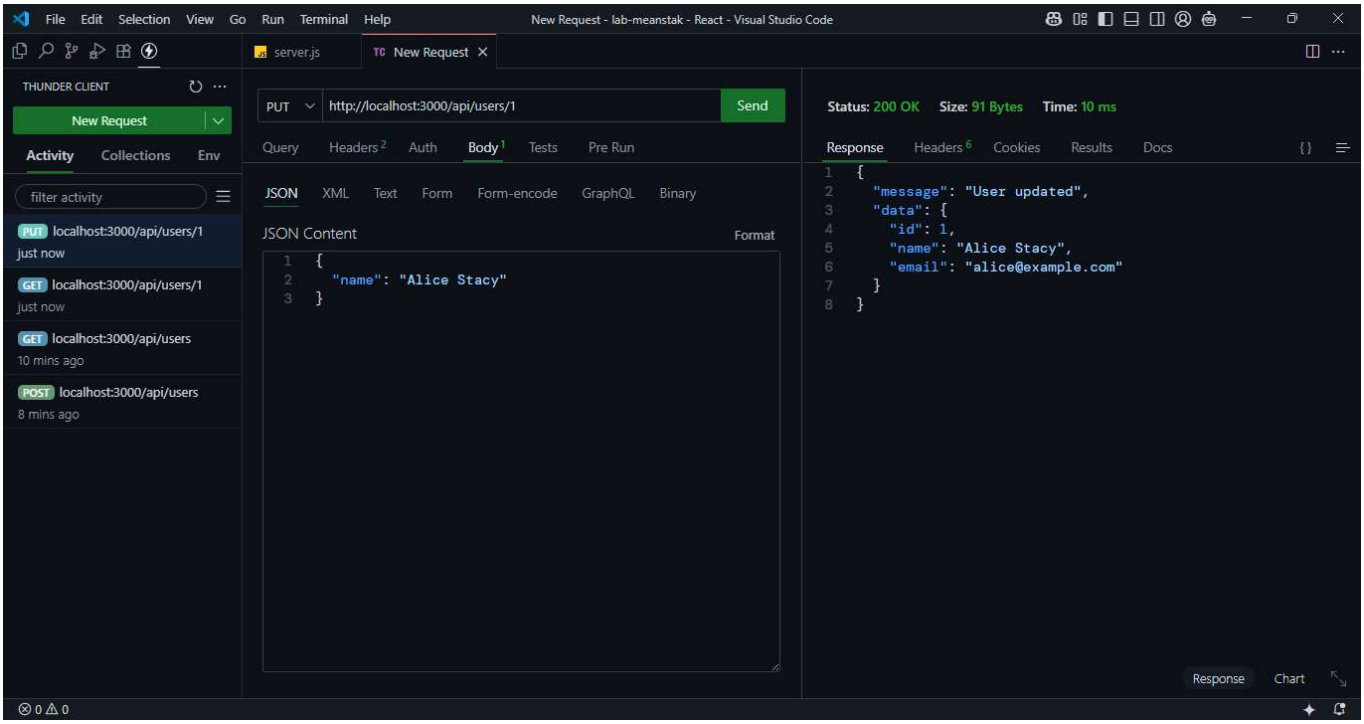
To GET Single User:



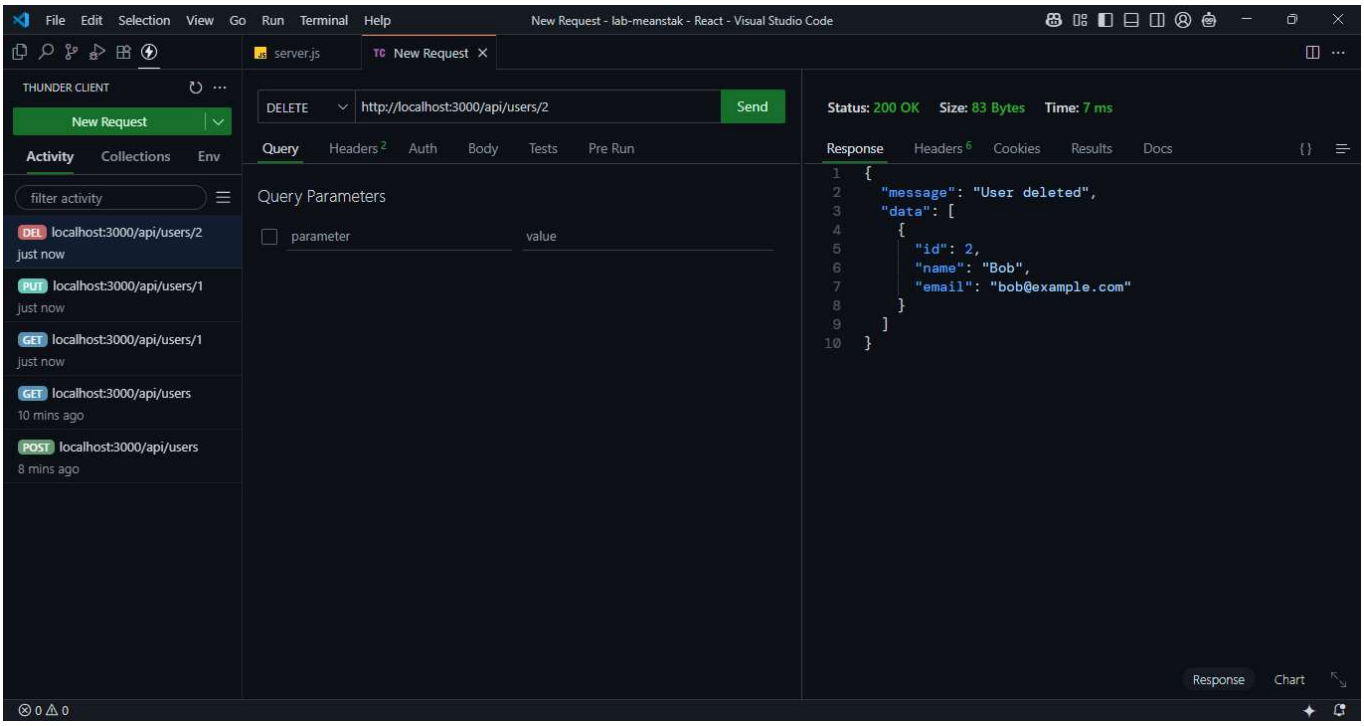
For POST request:



For PUT request:



For DELETE request:



Task 3: Connect Express App to MongoDB with CRUD

Aim:

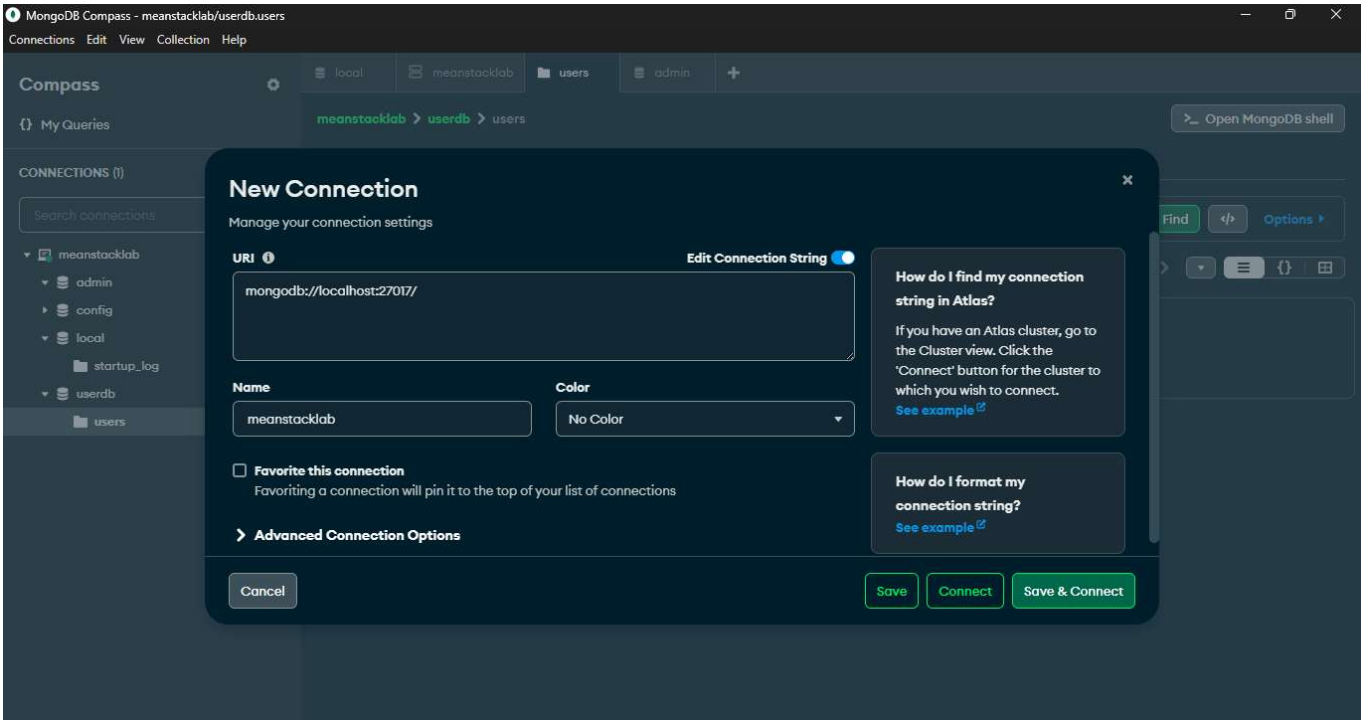
Develop an Express.js application that connects to a MongoDB database and implements basic CRUD (Create, Read, Update, Delete) operations for a resource (e.g., users).

Requirements:

- Node.js installed.
- MongoDB installed locally or accessible via MongoDB Atlas (cloud).
- Initialize a project with npm init -y.
- Install dependencies: npm install express mongoose.
- MongoDB server running (e.g., mongod for local or a MongoDB Atlas connection string).

Procedure:

1. **Set up MongoDB:**
    - Use a local MongoDB instance or create a free MongoDB Atlas cluster and obtain the connection string.
    - MongoDB Community Server here https://www.mongodb.com/try/download/community
    - Open MongoDB Compass.
    - Click **New Connection (+)**.
    - Enter: mongodb://localhost:27017



- Connect to the running instance.

2. **Configure Express app:**
   - Create an Express server and connect to MongoDB using Mongoose.
   - Define a schema and model for a "User" resource.
3. **Implement CRUD routes:**
   - Create routes for creating, reading, updating, and deleting users.
   - RESTful Routes:
     - **GET** /api/users: Returns the list of all users.
     - **GET** /api/users/:id: Returns a specific user by ID or a 404 error if not found.
     - **POST** /api/users: Creates a new user with a name provided in the request body.
     - **PUT** /api/users/:id: Updates the name of an existing user by ID.
     - **DELETE** /api/users/:id: Deletes a user by ID.
4. **Test the application:**
   - Run the server and test CRUD operations using Postman or curl.

Code:

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;

// Middleware to parse JSON
```

```
app.use(express.json());

// MongoDB connection
mongoose
   .connect('mongodb://127.0.0.1:27017/userdb', {
      useNewUrlParser: true,
      useUnifiedTopology: true,
   })
   .then(() => console.log('Connected to MongoDB'))
   .catch((err) => console.error('MongoDB connection error:', err));

// User schema and model
const userSchema = new mongoose.Schema({
   name: { type: String, required: true },
   email: { type: String, required: true, unique: true },
});
const User = mongoose.model('User', userSchema);

// CREATE: Add a new user
app.post('/api/users', async (req, res) => {
   try {
      const { name, email } = req.body;
      if (!name || !email)
         return res
            .status(400)
            .json({ message: 'Name and email are required' });
      const user = new User({ name, email });
      await user.save();
      res.status(201).json(user);
   } catch (err) {
      res.status(500).json({ message: err.message });
   }
});
```

```
// READ: Get all users
app.get('/api/users', async (req, res) => {
   try {
      const users = await User.find();
      res.json(users);
   } catch (err) {
      res.status(500).json({ message: err.message });
   }
});

// READ: Get a user by ID
app.get('/api/users/:id', async (req, res) => {
   try {
      const user = await User.findById(req.params.id);
      if (!user) return res.status(404).json({ message: 'User not found' });
      res.json(user);
   } catch (err) {
      res.status(500).json({ message: err.message });
   }
});

// UPDATE: Update a user by ID
app.put('/api/users/:id', async (req, res) => {
   try {
      const { name, email } = req.body;
      if (!name || !email)
         return res
            .status(400)
            .json({ message: 'Name and email are required' });
      const user = await User.findByIdAndUpdate(
         req.params.id,
         { name, email },
         { new: true, runValidators: true }
      );
```

```
        if (!user) return res.status(404).json({ message: 'User not found' });
        res.json(user);
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

// DELETE: Delete a user by ID
app.delete('/api/users/:id', async (req, res) => {
    try {
        const user = await User.findByIdAndDelete(req.params.id);
        if (!user) return res.status(404).json({ message: 'User not found' });
        res.json({ message: 'User deleted', user });
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

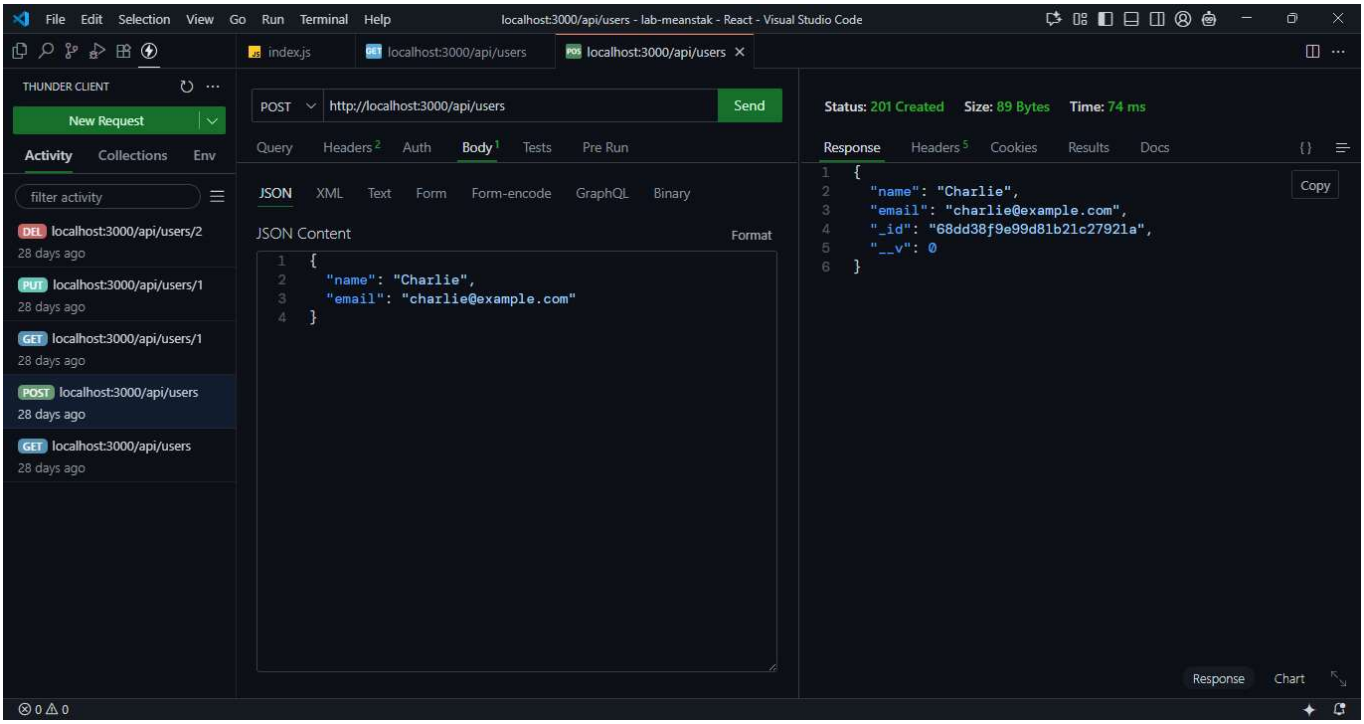
// Start the server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}/`);
});
```

```
Server running at http://localhost:3000/
Connected to MongoDB
```

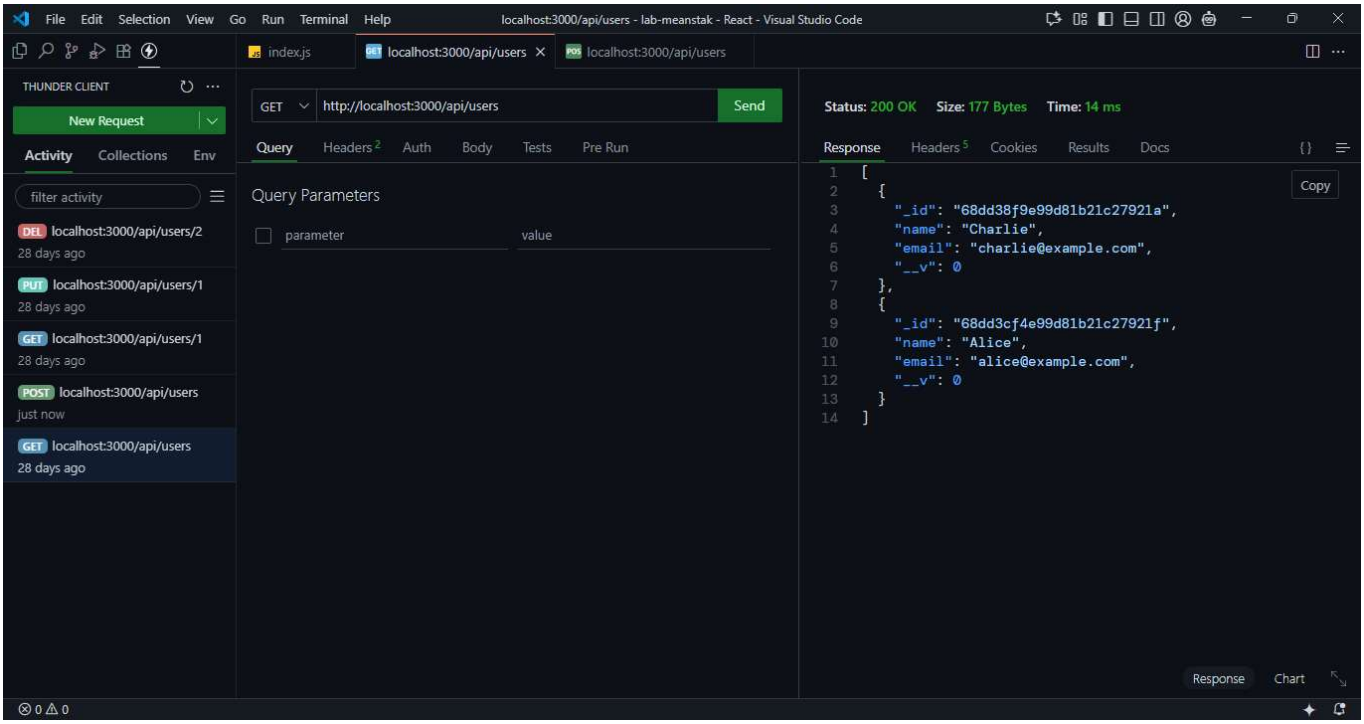
Output:

After installing dependencies and running node server.js, the server connects to MongoDB and listens on port 3000.

For PUT request:

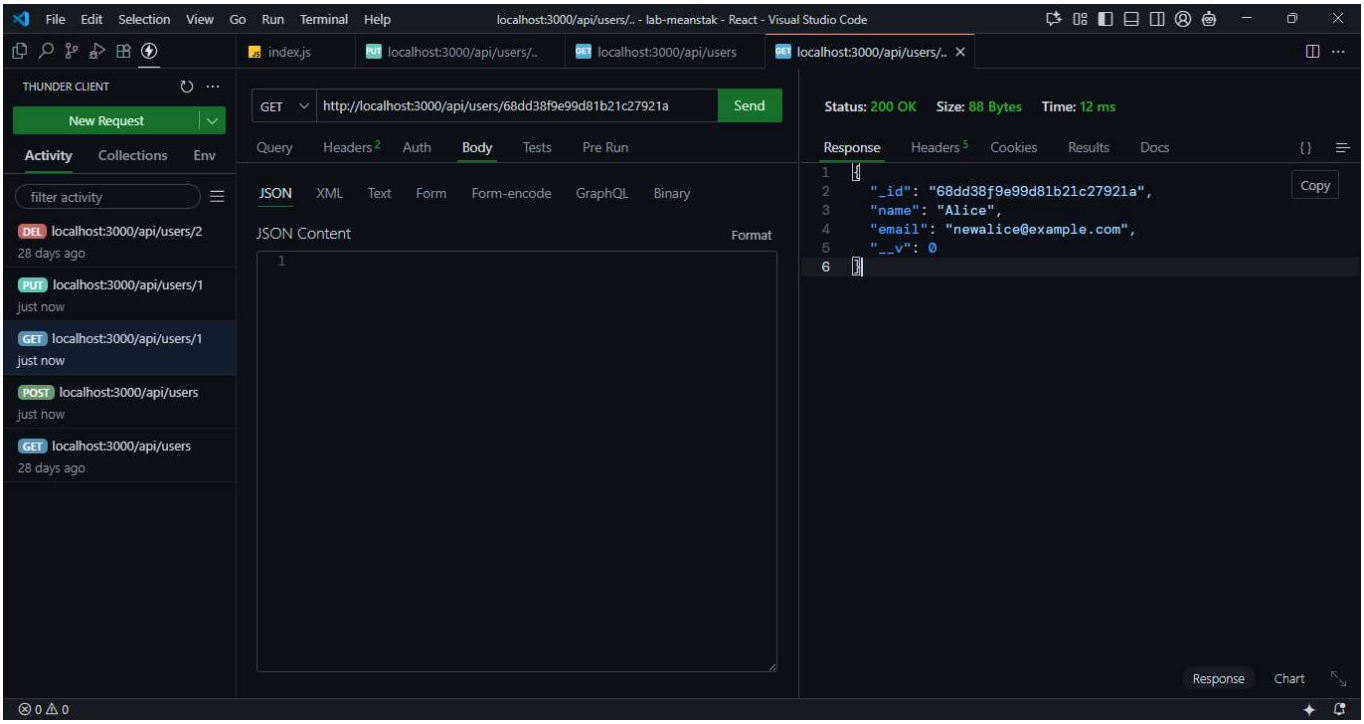


For GET request:

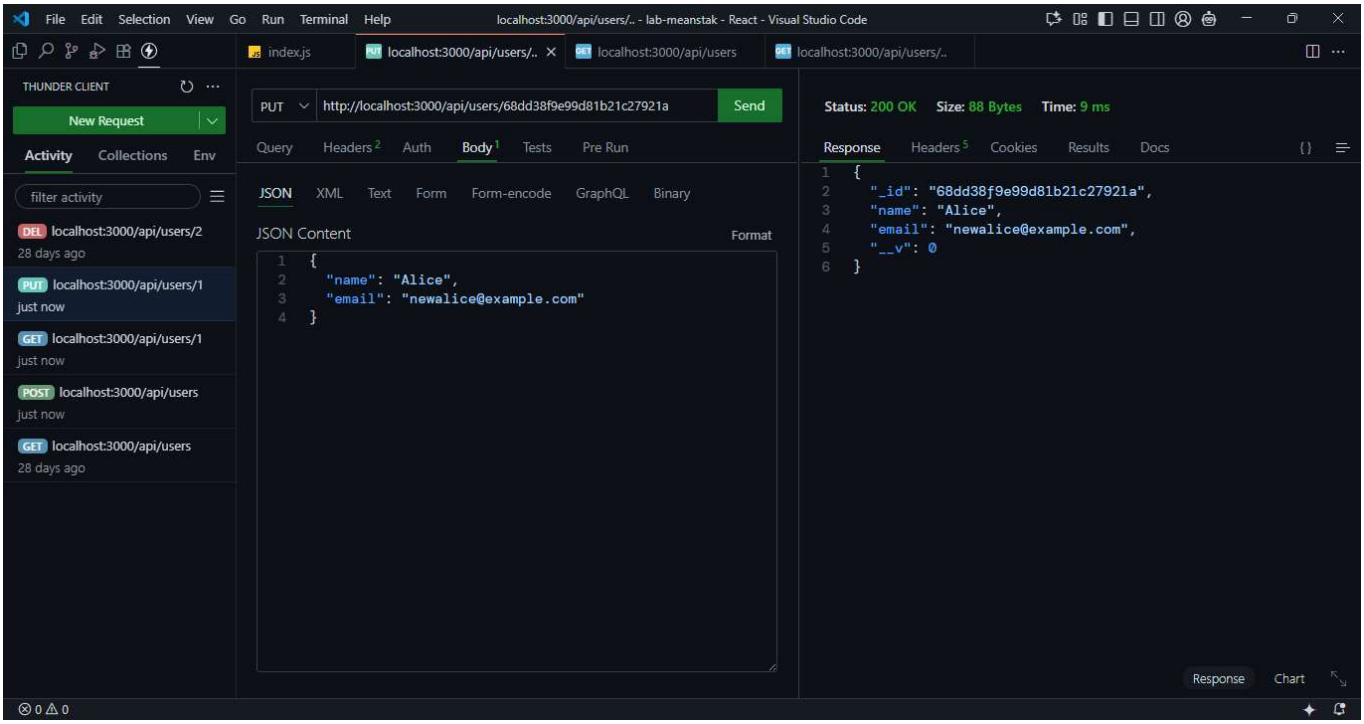


For GET Single User:

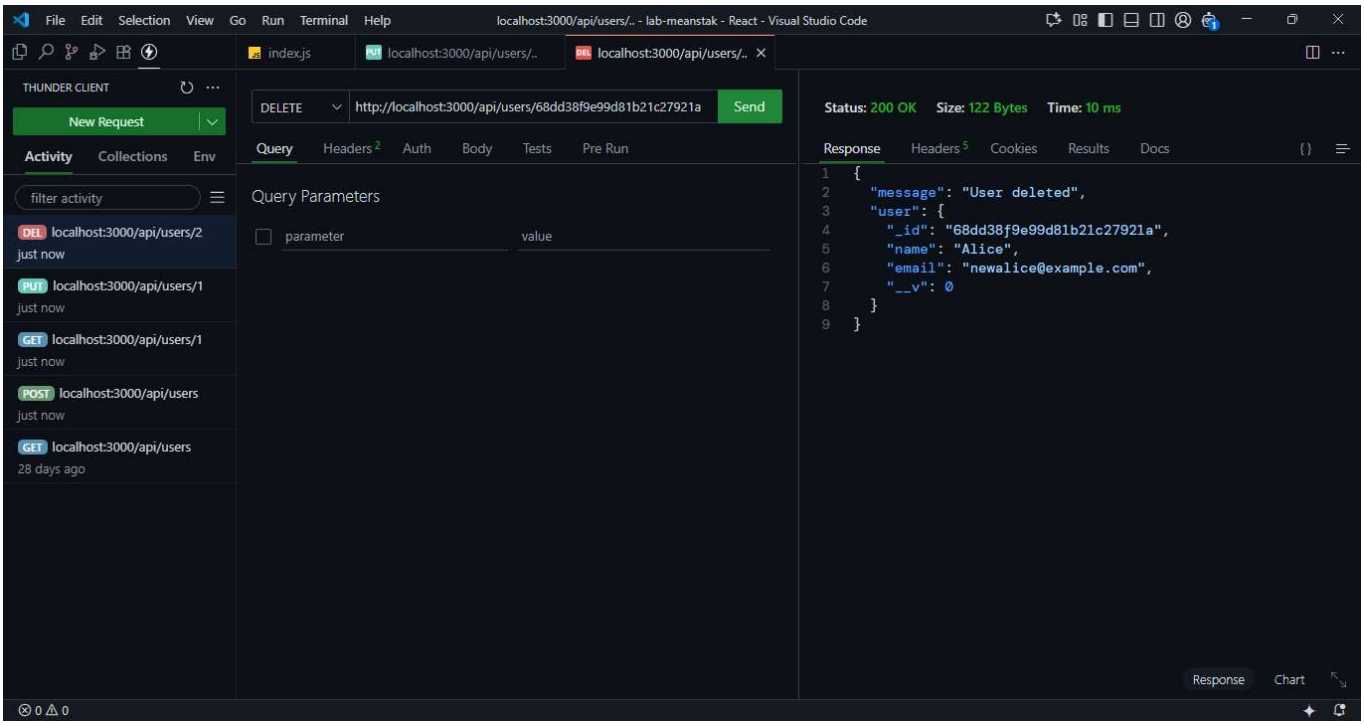
Enter http://localhost:3000/api/users/:id, replacing :id with the MongoDB _id of the user.



For PUT request:



For DELETE request:



Task 4: Implement Session Management Using Cookies and Helmet Middleware

Aim:

Develop an Express.js application that implements session management using cookies and secures the app with Helmet middleware.

Requirements:

- Node.js installed.
- Initialize a project with npm init -y.
- Install dependencies: npm install express express-session cookie-parser helmet.

Procedure:

1. **Set up Express app:**
   - Create an Express server with session management using express-session and cookie-parser.
   - Secure the app using helmet middleware.
2. **Implement session management:**
   - Create routes to demonstrate session handling (e.g., login, access protected resource, logout).
   - Use cookies to track user sessions.
3. **Secure with Helmet:**
   - Apply Helmet middleware to set security-related HTTP headers.
4. **Test the application:**

- Run the server and test session management and security headers using a browser or Postman.

Code:

```
const express = require('express');
const session = require('express-session');
const cookieParser = require('cookie-parser');
const helmet = require('helmet');
const app = express();
const PORT = 3000;

// Middleware setup
app.use(express.json());
app.use(cookieParser());
app.use(helmet()); // Adds security headers
app.use(
   session({
      secret: 'your-secret-key', // Replace with a strong secret in
production
      resave: false,
      saveUninitialized: false,
      cookie: {
         secure: false, // Set to true in production with HTTPS
         httpOnly: true, // Prevents client-side access to cookies
         maxAge: 24 * 60 * 60 * 1000, // 1 day in milliseconds
      },
   })
);

// Simulated user database
const users = [{ id: 1, username: 'alice', password: 'password123' }];

// Login route
app.post('/api/login', (req, res) => {
   const { username, password } = req.body;
```

```
    const user = users.find(
        (u) => u.username === username && u.password === password
    );
    if (!user) return res.status(401).json({ message: 'Invalid
credentials' });

    // Store user info in session
    req.session.user = { id: user.id, username: user.username };
    res.json({ message: 'Login successful', user: req.session.user });
});

// Protected route
app.get('/api/protected', (req, res) => {
    if (!req.session.user)
        return res.status(401).json({ message: 'Unauthorized' });
    res.json({
        message: 'Access granted to protected resource',
        user: req.session.user,
    });
});

// Logout route
app.post('/api/logout', (req, res) => {
    req.session.destroy((err) => {
        if (err) return res.status(500).json({ message: 'Logout failed' });
        res.clearCookie('connect.sid'); // Clear session cookie
        res.json({ message: 'Logout successful' });
    });
});

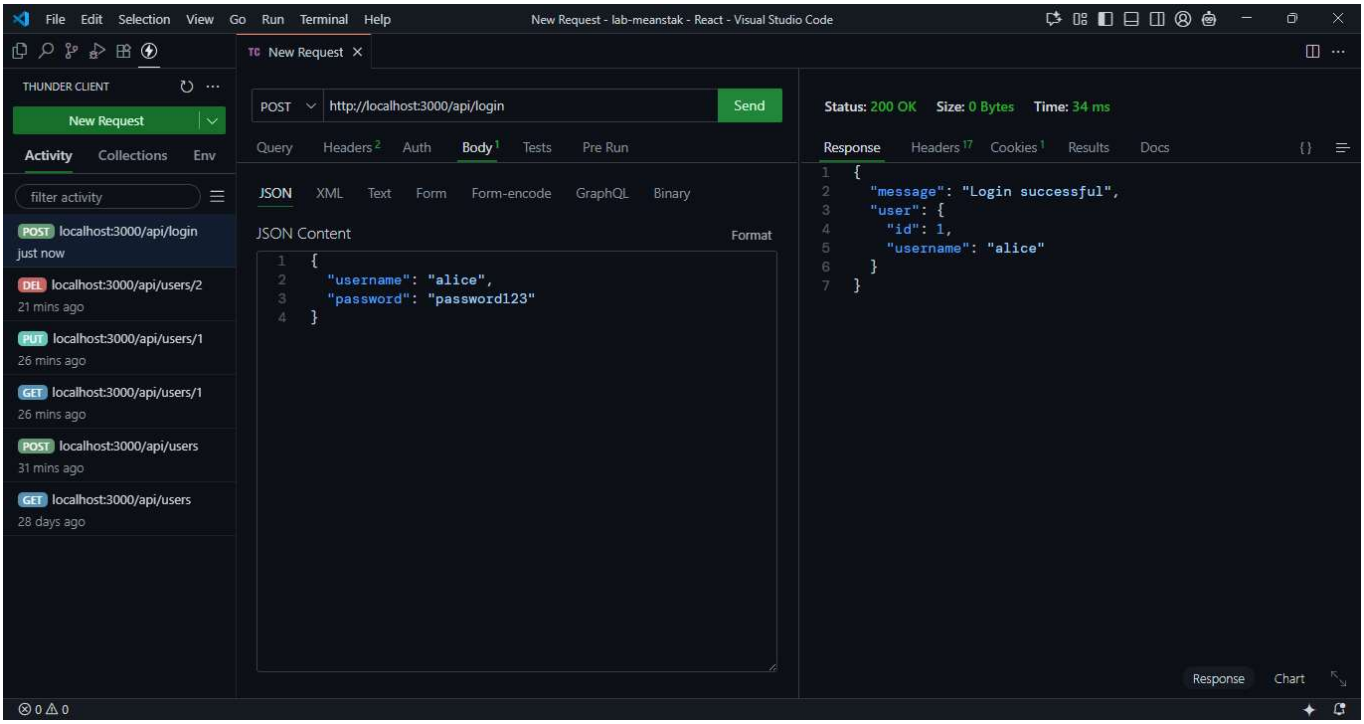
// Start the server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}/`);
});
```

```
@Maharshi →node(22.17.1) ~\OneDrive\Desktop\lab-meanstak →bun index.js
Server running at http://localhost:3000/
```

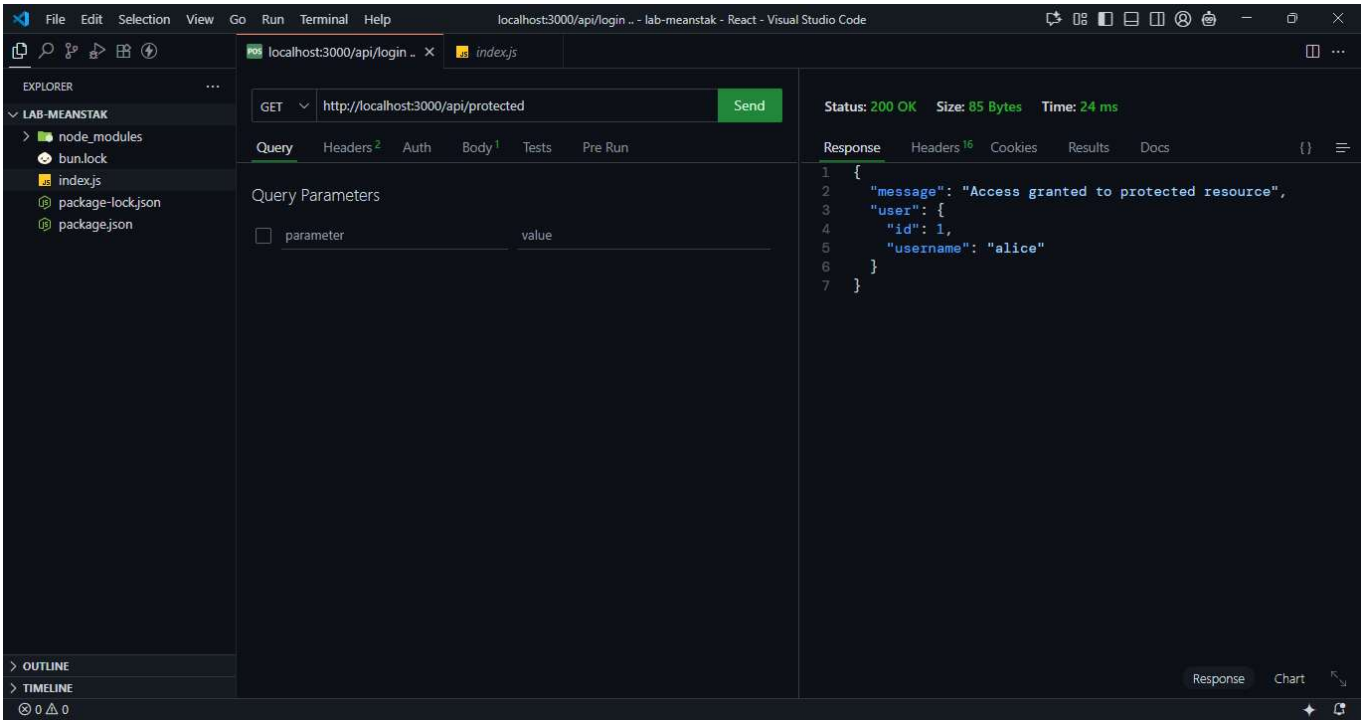
Output:

- After installing dependencies and running `node app.js`, the server starts on port 3000 with session management and security headers enabled.
- **Session Management:**
  - **POST** `/api/login`: Authenticates a user (e.g., `{"username":"alice","password":"password123"}`) and creates a session, storing user data in req.session. A session cookie (`connect.sid`) is set in the browser.
  - **GET** `/api/protected`: Checks for an active session; returns user data if authenticated, else a 401 error.
  - **POST** `/api/logout`: Destroys the session and clears the cookie.
- Helmet Middleware: Adds headers like `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, and others to enhance security.

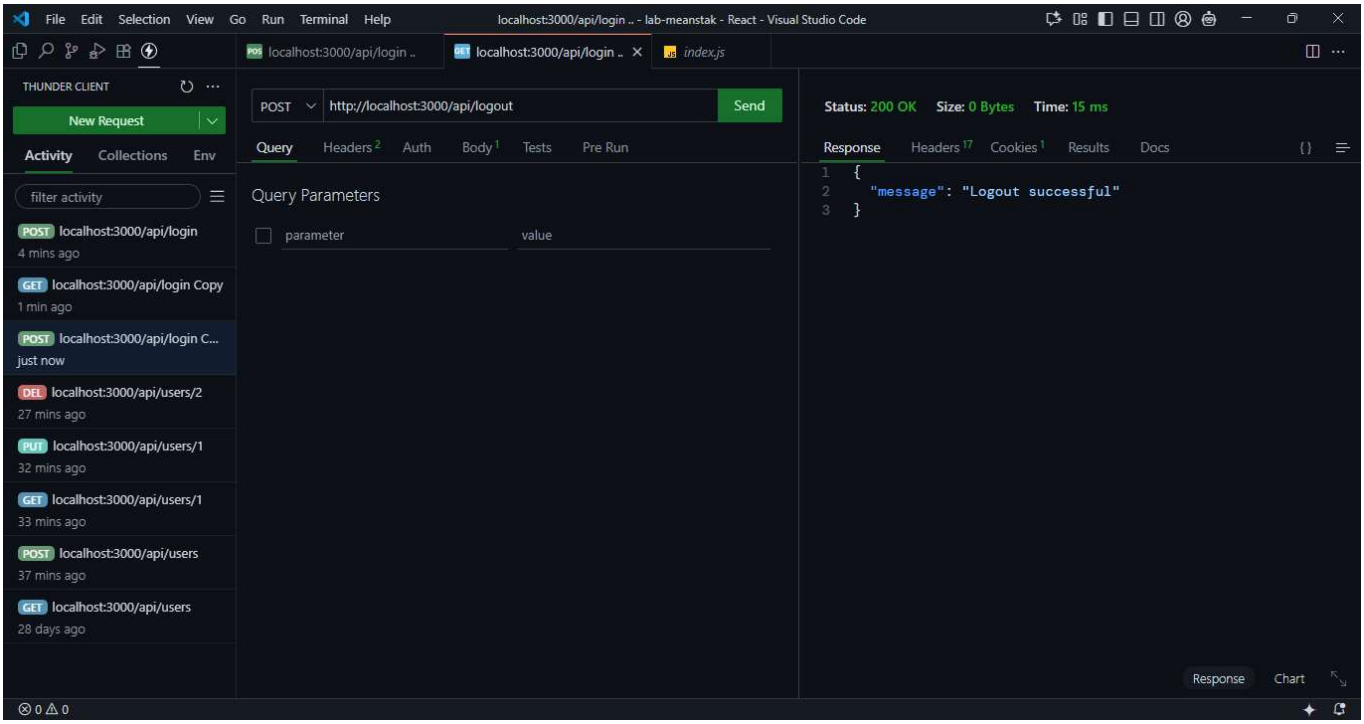
`POST /api/login` Request:



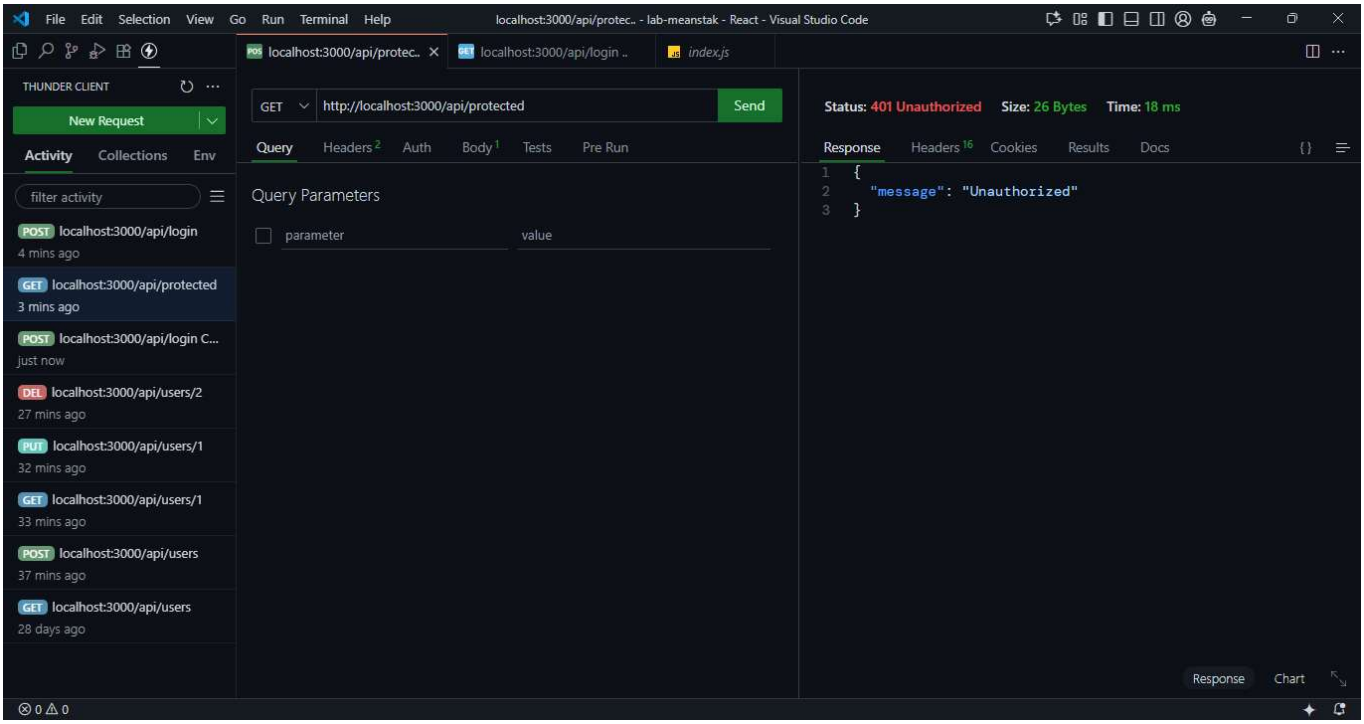
`POST /api/protected` (After Authentication) Request:



POST /api/logout Request:



POST /api/protected (Without Authentication) Request:



UNIT III: MongoDB & TypeScript

MongoDB: CRUD, indexing, aggregation, collections. TypeScript: Interfaces, Classes, Generics, Modules.

Tasks:

1. Install MongoDB and create a cluster using MongoDB Atlas.
2. Write MongoDB CRUD queries to insert, update, delete documents.
3. Create a TypeScript app using interfaces, classes, and functions,
4. Build a module-based project in TypeScript connecting to a MongoDB collection.

Task 1: Install MongoDB and Create a Cluster Using MongoDB Atlas

Aim:

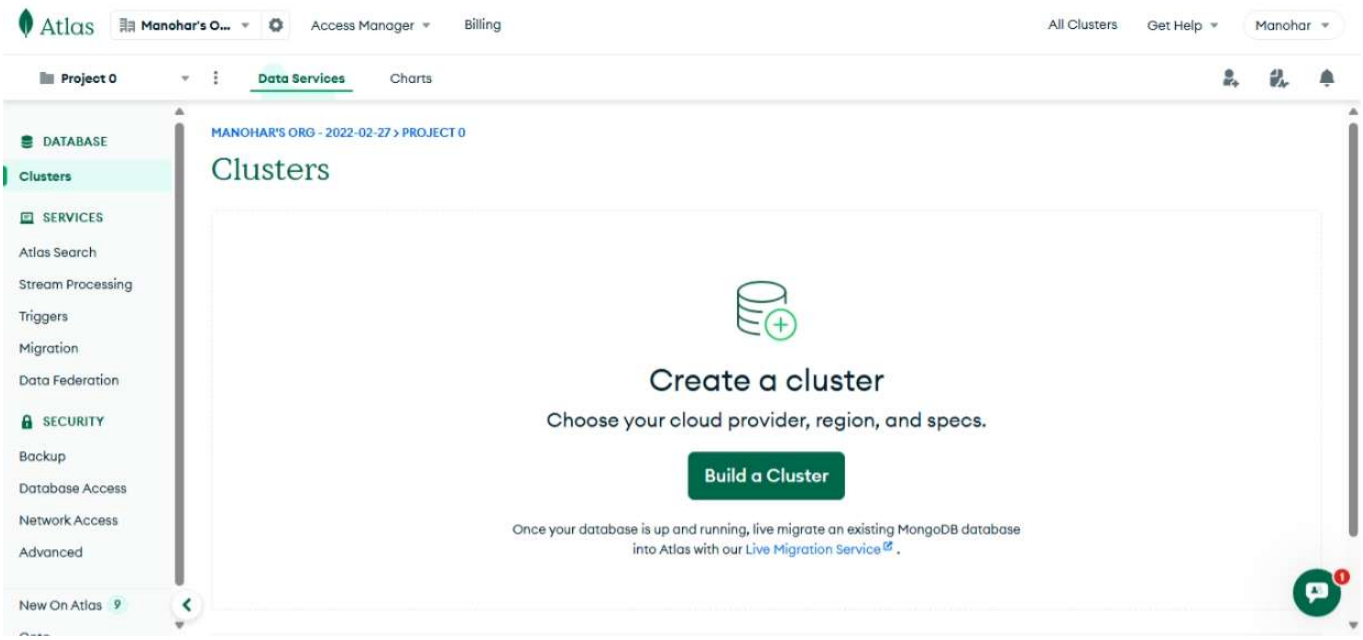
Set up MongoDB locally and create a cloud-based MongoDB Atlas cluster to host a database for application development.

Requirements:

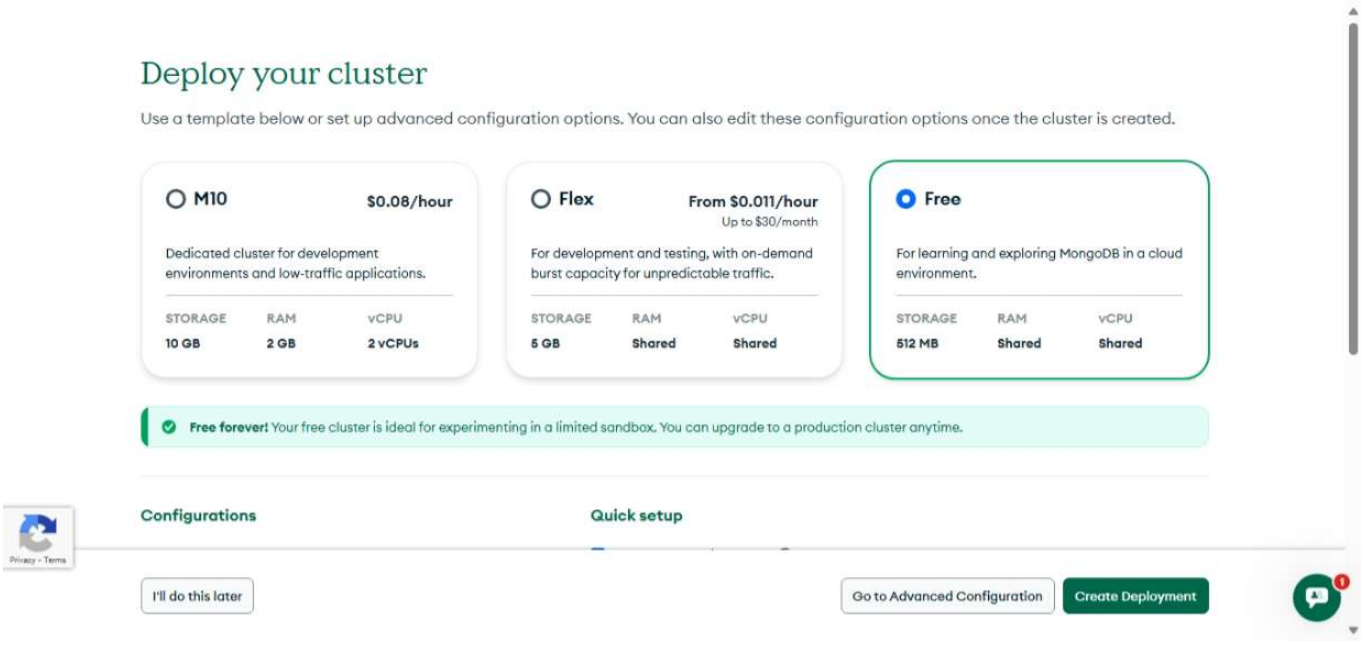
- A computer with internet access.
- Administrative privileges for local MongoDB installation.
- A MongoDB Atlas account (free tier available).
- Web browser for accessing MongoDB Atlas.
- Optional: MongoDB Compass or a terminal for local MongoDB interaction.

Procedure:

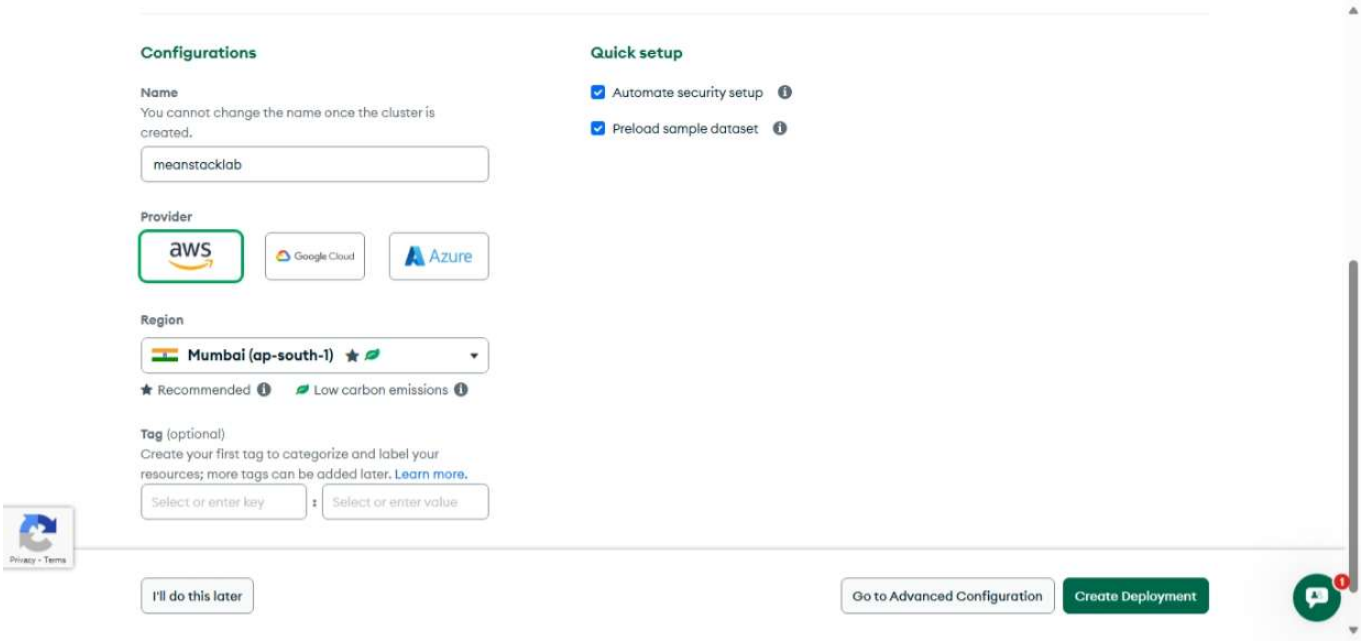
- Install MongoDB Locally:
  - Download and install MongoDB Shell from here MongoDB Shell Download.
- **Sign Up for MongoDB Atlas:**
  - Go to MongoDB Atlas https://www.mongodb.com/products/platform/atlas-database.
  - Create an account or log in.
- **Create a Cluster**:
  - Click "Build a Cluster" and select the "Shared" (free) tier.



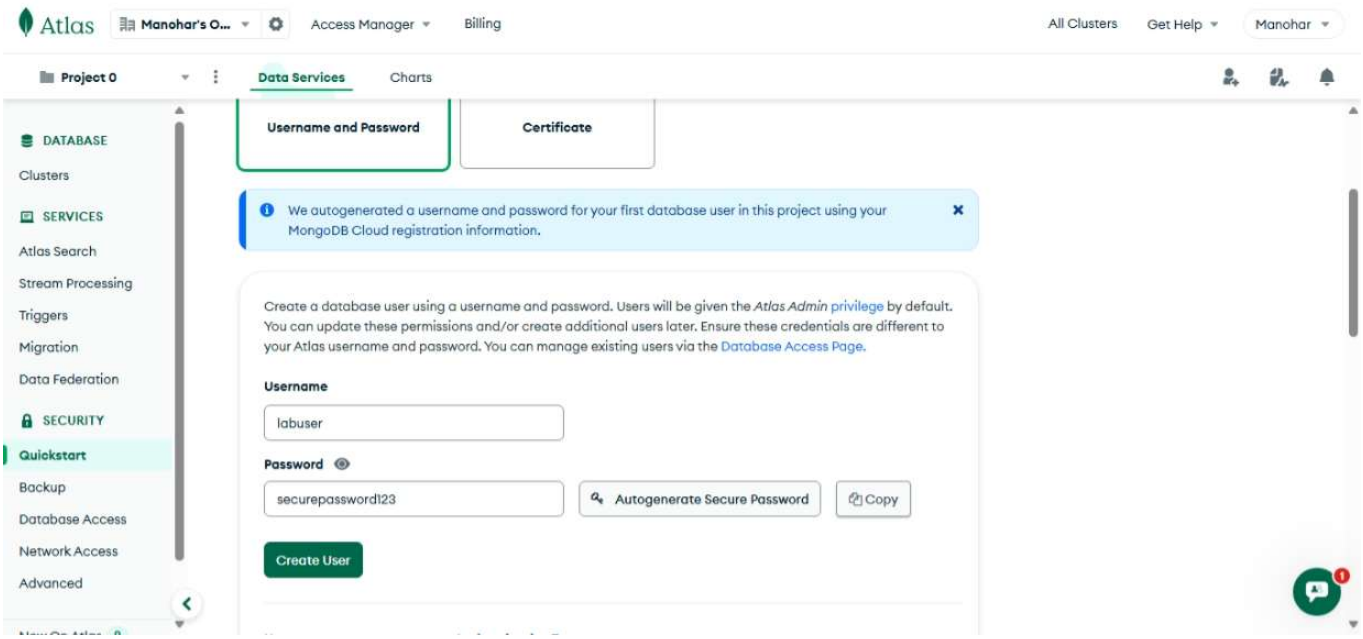
  - Choose a cloud provider (e.g., AWS) and region.



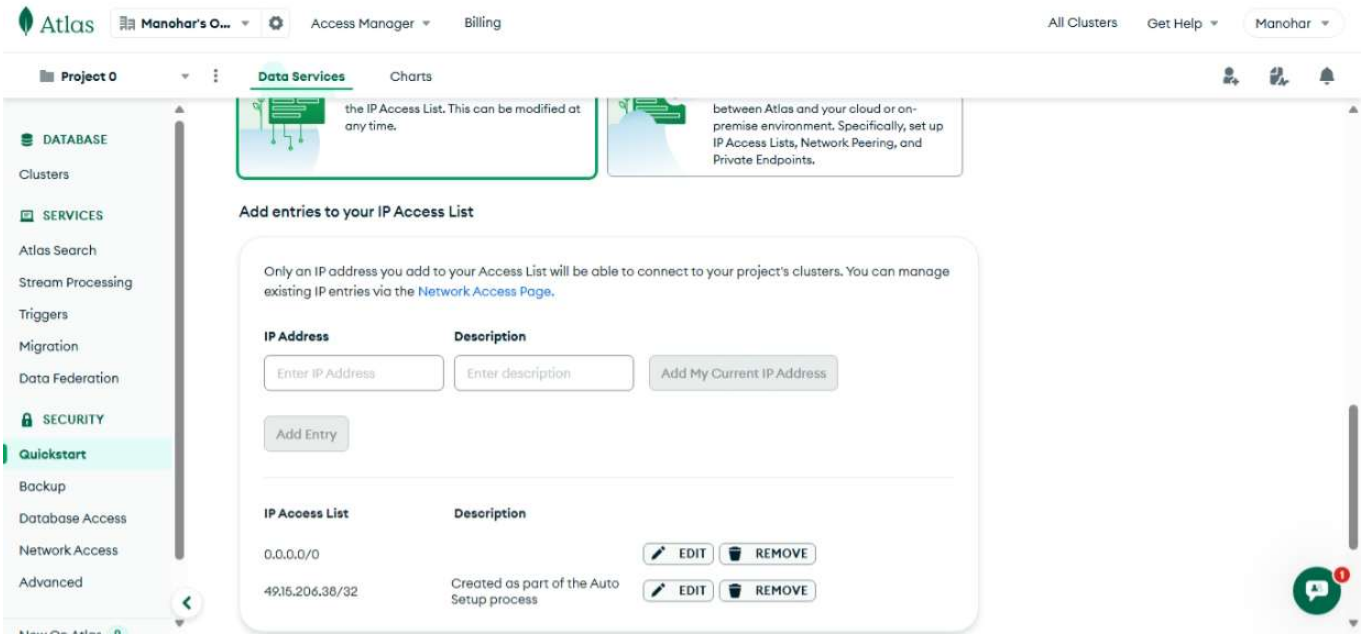
- Name the cluster (e.g., meanstacklab) and click "Create Cluster".



- **Configure Cluster:**
  - Database Access: Add a database user (e.g., username: labuser, password: securepassword123).

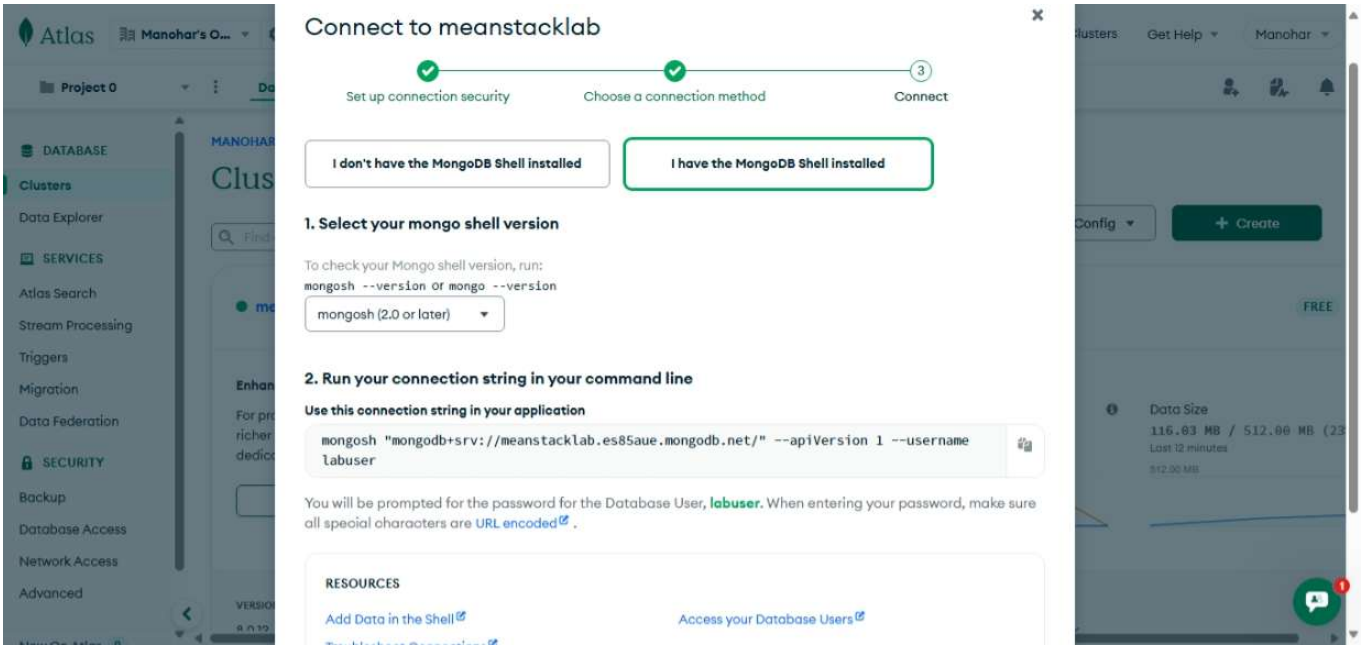


  - Network Access: Add IP address 0.0.0.0/0 (for testing; restrict in production).



- Connect: Click "Connect", select "Connect your application", and copy the connection string (e.g., mongodb+srv://labuser:<password>@labcluster.mongodb.net/test)

- **Test Atlas Connection:**
  - Use MongoDB Compass or a terminal with mongosh:



  - Copy the connection string and paste it in the terminal.
  - Run db.runCommand({ ping: 1 }) to verify (expected output: { ok: 1 }).

```
@Maharshi → ~\OneDrive\Desktop → mongosh "mongodb+srv://meanstacklab.es85aue.mongodb.net/" --apiVersion 1 --username labuser
Enter password: ****************
Current Mongosh Log ID: 68dd4e0da0efa84521cebea3
Connecting to:          mongodb+srv://<credentials>@meanstacklab.es85aue.mongodb.net/?appName=mongosh+2.5.8
Using MongoDB:          8.0.13 (API Version 1)
Using Mongosh:          2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

Atlas atlas-o3hvyp-shard-0 [primary] test> db.runCommand({ ping: 1 })
{ ok: 1 }
Atlas atlas-o3hvyp-shard-0 [primary] test>
```

```
use testdb;
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] test> use testdb;
switched to db testdb
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

```
// CREATE: Insert documents into the users collection
db.users.insertMany([
  { name: "Alice", email: "alice@example.com", age: 25 },
  { name: "Bob", email: "bob@example.com", age: 30 }
]);
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] test> use testdb;
switched to db testdb
Atlas atlas-o3hvyp-shard-0 [primary] testdb> // CREATE: Insert documents into the users collection
... db.users.insertMany([
...   { name: "Alice", email: "alice@example.com", age: 25 },
...   { name: "Bob", email: "bob@example.com", age: 30 }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68dd520745e4bcf7b6cebea4'),
    '1': ObjectId('68dd520745e4bcf7b6cebea5')
  }
}
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Query:

```
// READ: Query all documents
db.users.find().pretty();
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> db.users.find().pretty();
[
  {
    _id: ObjectId('68dd520745e4bcf7b6cebea4'),
    name: 'Alice',
    email: 'alice@example.com',
    age: 25
  },
  {
    _id: ObjectId('68dd520745e4bcf7b6cebea5'),
    name: 'Bob',
    email: 'bob@example.com',
    age: 30
  }
]
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Query:

```
// READ: Query a specific document by email
db.users.findOne({ email: "alice@example.com" });
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> db.users.findOne({ email: "alice@example.com" });
{
  _id: ObjectId('68dd520745e4bcf7b6cebea4'),
  name: 'Alice',
  email: 'alice@example.com',
  age: 25
}
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Query:

```
// UPDATE: Update a single document (e.g., change Alice's age)
db.users.updateOne(
  { email: "alice@example.com" },
  { $set: { age: 26 } }
);
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> // UPDATE: Update a single document (e.g., change Alice's age)
... db.users.updateOne(
...   { email: "alice@example.com" },
...   { $set: { age: 26 } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Query:

```
// UPDATE: Update multiple documents (e.g., add a field to all users)
db.users.updateMany(
  {},
  { $set: { status: "active" } }
);
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> db.users.updateMany(
...   {},
...   { $set: { status: "active" } }
... );
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> db.users.find().pretty();
[
  {
    _id: ObjectId('68dd520745e4bcf7b6cebea4'),
    name: 'Alice',
    email: 'alice@example.com',
    age: 26,
    status: 'active'
  },
  {
    _id: ObjectId('68dd520745e4bcf7b6cebea5'),
    name: 'Bob',
    email: 'bob@example.com',
    age: 30,
    status: 'active'
  }
]
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Query:

```
// DELETE: Delete a single document by email
```

```
db.users.deleteOne({ email: "bob@example.com" });
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> db.users.deleteOne({ email: "bob@example.com" });
{ acknowledged: true, deletedCount: 1 }
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Query:

```
// DELETE: Delete multiple documents (e.g., users with age > 25)
db.users.deleteMany({ age: { $gt: 25 } });
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> db.users.deleteMany({ age: { $gt: 25 } });
{ acknowledged: true, deletedCount: 1 }
Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Query:

```
// READ: Verify the collection after modifications
db.users.find().pretty();
```

Output:

```
Atlas atlas-o3hvyp-shard-0 [primary] testdb> db.users.find().pretty();

Atlas atlas-o3hvyp-shard-0 [primary] testdb>
```

Output:

Setup: Connected to testdb database using mongosh (e.g., mongosh mongodb://localhost:27017/testdb or Atlas connection string).

CRUD Operations:

- **Create:** Inserted two documents (Alice and Bob) into the users collection. Output: { acknowledged: true, insertedIds: { ... } }.
- **Read:** find() returns all documents (e.g., [{ name: "Alice", email: "alice@example.com", age: 25 }, ...]); findOne returns Alice's document.
- **Update:** Updated Alice's age to 26; added status: "active" to all documents. Output: { modifiedCount: 1 } for updateOne, { modifiedCount: 2 } for updateMany.
- **Delete:** Removed Bob's document; deleted documents with age > 25. Output: { deletedCount: 1 } or { deletedCount: N }.

Testing: Run queries in mongosh or Compass. Final find() shows remaining documents (e.g., none or updated documents depending on deletions).

Task 3: Create a TypeScript App Using Interfaces, Classes, and Functions

Aim:

Develop a TypeScript application that demonstrates the use of interfaces, classes, and functions to manage a simple data structure (e.g., a user management system).

Requirements:

- Node.js installed.
- TypeScript installed globally (`npm install -g typescript`).
- Initialize a project with `npm init -y` and install TypeScript locally (`npm install typescript --save-dev`).
- A code editor (e.g., VS Code) for writing and compiling TypeScript code.

Procedure:

1. **Set up TypeScript project:**
    - Create a project directory and initialize it.
    - Run `npx tsc --init` in the project folder to Configure TypeScript with a `tsconfig.json` file.

2. **Define interfaces and classes:**
    - Create an interface for a user.
    - Implement a class to manage user data with methods for CRUD-like operations.
3. **Write functions:**
    - Add utility functions to process user data.
4. **Compile and run:**
    - Compile TypeScript to JavaScript and execute the application.

Code:

```
// Interface for User
interface User {
   id: number;
   name: string;
   email: string;
}

// Class to manage users
class UserManager {
   private users: User[] = [];
```

```
    // Add a new user
    addUser(user: User): void {
        this.users.push(user);
        console.log(`Added user: ${user.name}`);
    }

    // Get all users
    getUsers(): User[] {
        return this.users;
    }

    // Find a user by ID
    getUserById(id: number): User | undefined {
        return this.users.find((user) => user.id === id);
    }

    // Update a user
    updateUser(id: number, updatedData: Partial<User>): boolean {
        const user = this.getUserById(id);
        if (!user) return false;
        Object.assign(user, updatedData);
        return true;
    }

    // Delete a user
    deleteUser(id: number): boolean {
        const index = this.users.findIndex((user) => user.id === id);
        if (index === -1) return false;
        this.users.splice(index, 1);
        return true;
    }
}

// Function to display user info
```

```
function displayUser(user: User | undefined): string {
    if (!user) return 'User not found';
    return `ID: ${user.id}, Name: ${user.name}, Email: ${user.email}`;
}

// Main function to demonstrate usage
function main(): void {
    const userManager = new UserManager();

    // Create users
    const user1: User = { id: 1, name: 'Alice', email: 'alice@example.com' };
    const user2: User = { id: 2, name: 'Bob', email: 'bob@example.com' };

    // Add users
    userManager.addUser(user1);
    userManager.addUser(user2);

    // Display all users
    console.log('All Users:', userManager.getUsers());

    // Get a user by ID
    console.log('Find User:', displayUser(userManager.getUserById(1)));

    // Update a user
    userManager.updateUser(1, { name: 'Alice Updated' });
    console.log('After Update:', displayUser(userManager.getUserById(1)));

    // Delete a user
    userManager.deleteUser(2);
    console.log('After Delete:', userManager.getUsers());
}

// Run the application
main();
```

Functionality:

- **Interface:** User defines the structure for user objects.
- **Class:** UserManager handles CRUD operations (add, get, update, delete) on an in-memory user array.
- **Function:** displayUser formats user data for output.
- **Main:** Demonstrates creating, reading, updating, and deleting users.

Output:

- Compile with `npx tsc` to generate app.js.
- Run with `node app.js`.

```
@Maharshi ➜ node(22.17.1) ~\OneDrive\Desktop\task3 ➜ node app.js
Added user: Alice
Added user: Bob
All Users: [
  { id: 1, name: 'Alice', email: 'alice@example.com' },
  { id: 2, name: 'Bob', email: 'bob@example.com' }
]
Find User: ID: 1, Name: Alice, Email: alice@example.com
After Update: ID: 1, Name: Alice Updated, Email: alice@example.com
After Delete: [ { id: 1, name: 'Alice Updated', email: 'alice@example.com' } ]
@Maharshi ➜ node(22.17.1) ~\OneDrive\Desktop\task3 ➜
```

Task 4: Build a Module-Based TypeScript Project Connecting to a MongoDB Collection

Aim:

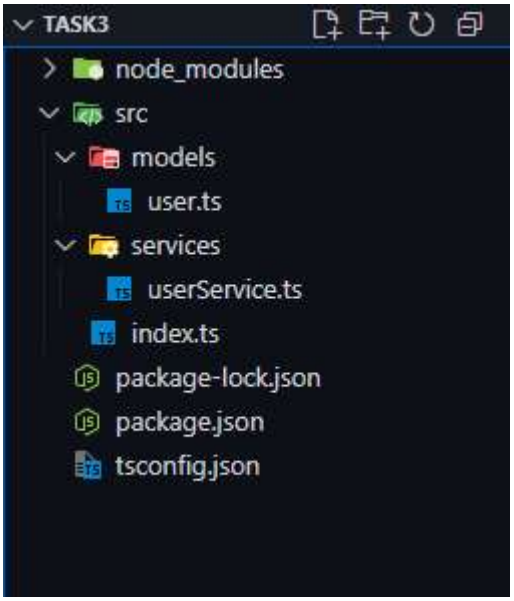
Develop a TypeScript application using modules, interfaces, and classes to perform CRUD operations on a MongoDB collection.

Requirements:

- Node.js and TypeScript installed (`npm install -g typescript`).
- MongoDB installed locally or a MongoDB Atlas cluster set up (from Task 1).
- Initialize a project with `npm init -y`.
- Install dependencies: `npm install typescript ts-node @types/node @types/mongoose mongoose --save-dev`.
- MongoDB server running or Atlas connection string available.
- A code editor (e.g., VS Code) and tsconfig.json configured.

Code:

Folder Structure:



In tsconfig.json

```
{
   "compilerOptions": {
      "target": "es6",
      "module": "commonjs",
      "outDir": "./dist",
      "rootDir": "./src",
      "strict": true,
      "esModuleInterop": true,
      "skipLibCheck": true
   },
   "include": ["src/**/*"],
   "exclude": ["node_modules"]
}
```

In src/models/user.ts

```
import mongoose, { Document, Schema } from 'mongoose';

export interface IUser extends Document {
   name: string;
   email: string;
   age: number;
}
```

```
const UserSchema = new Schema<IUser>({
   name: { type: String, required: true },
   email: { type: String, required: true, unique: true },
   age: { type: Number, required: true },
});

export const User = mongoose.model<IUser>('User', UserSchema);
```

In src/services/userService.ts

```
import { IUser, User } from '../models/user';

export class UserService {
   async createUser(data: Partial<IUser>): Promise<IUser> {
      const user = new User(data);
      return user.save();
   }

   async getUsers(): Promise<IUser[]> {
      return User.find();
   }

   async getUserById(id: string): Promise<IUser | null> {
      return User.findById(id);
   }

   async updateUser(id: string, data: Partial<IUser>): Promise<IUser | null>
{
      return User.findByIdAndUpdate(id, data, { new: true });
   }

   async deleteUser(id: string): Promise<IUser | null> {
      return User.findByIdAndDelete(id);
   }
}
```

In src/index.ts

```
import mongoose from 'mongoose';
import { UserService } from './services/userService';

async function main() {
   // Connect to MongoDB
   await mongoose.connect('mongodb://127.0.0.1:27017/testdb');
   console.log('Connected to MongoDB');

   const userService = new UserService();

   // Create users
   const user1 = await userService.createUser({
      name: 'Alice',
      email: 'alice@example.com',
      age: 25,
   });
   console.log('Created User:', user1);

   const user2 = await userService.createUser({
      name: 'Bob',
      email: 'bob@example.com',
      age: 30,
   });
   console.log('Created User:', user2);

   // Read all users
   const users = await userService.getUsers();
   console.log('All Users:', users);

   // Read a user by ID
   const user = await userService.getUserById(user1.id);
   console.log('User by ID:', user);
```

```
    // Update a user
    const updatedUser = await userService.updateUser(user1.id, {
        name: 'Alice Updated',
        age: 26,
    });
    console.log('Updated User:', updatedUser);

    // Delete a user
    const deleted = await userService.deleteUser(user2.id);
    console.log('Deleted User:', deleted);

    // Read all users after deletion
    const remainingUsers = await userService.getUsers();
    console.log('Remaining Users:', remainingUsers);

    // Close MongoDB connection
    await mongoose.connection.close();
}

main().catch((err) => console.error('Error:', err));
```

Functionality:

- **Modules:** user.ts defines the Mongoose schema and model; userService.ts encapsulates CRUD logic; index.ts orchestrates operations.
- **Interface:** IUser ensures type safety for user data.
- **Class:** UserService handles MongoDB CRUD operations asynchronously.
- **MongoDB:** Connects to testdb, performs CRUD on the users collection, and closes the connection.

Execution:

Run `npx ts-node src/index.ts`.

Testing: Use MongoDB Compass or mongosh to verify the users collection in testdb. Check for inserted, updated, and deleted documents.

Output:

```
@Maharshi ➜ node(22.17.1) ~\OneDrive\Desktop\task3 ➜ npx ts-node
src/index.ts
Connected to MongoDB
Created User: {
  name: 'Alice',
  email: 'alice@example.com',
  age: 25,
  _id: new ObjectId('68dd650c55b1372176e81a48'),
  __v: 0
}
Created User: {
  name: 'Bob',
  email: 'bob@example.com',
  age: 30,
  _id: new ObjectId('68dd650c55b1372176e81a4b'),
  __v: 0
}
All Users: [
  {
    _id: new ObjectId('68dd650c55b1372176e81a48'),
    name: 'Alice',
    email: 'alice@example.com',
    age: 25,
    __v: 0
  },
  {
    _id: new ObjectId('68dd650c55b1372176e81a4b'),
    name: 'Bob',
    email: 'bob@example.com',
    age: 30,
    __v: 0
  }
]
```

```
User by ID: {
  _id: new ObjectId('68dd650c55b1372176e81a48'),
  name: 'Alice',
  email: 'alice@example.com',
  age: 25,
  __v: 0
}
Updated User: {
  _id: new ObjectId('68dd650c55b1372176e81a48'),
  name: 'Alice Updated',
  email: 'alice@example.com',
  age: 26,
  __v: 0
}
Deleted User: {
  _id: new ObjectId('68dd650c55b1372176e81a4b'),
  name: 'Bob',
  email: 'bob@example.com',
  age: 30,
  __v: 0
}
Remaining Users: [
  {
    _id: new ObjectId('68dd650c55b1372176e81a48'),
    name: 'Alice Updated',
    email: 'alice@example.com',
    age: 26,
    __v: 0
  }
]
```

UNIT IV: Angular Basics & Component Development

Angular setup, Components, Templates, Data Binding.Directives (@if, @for, @switch), Pipes, Forms.

1. Create a new Angular component and display content using interpolation.
2. Implement @if and @for to dynamically show/hide elements and list data.
3. Build a login form with validation using a template-driven approach.
4. Use built-in pipes to transform data (uppercase, lowercase, date).

Task 1: Create a New Angular Component and Display Content Using Interpolation

Aim:

Set up a new Angular project and create a component that displays dynamic content using interpolation (e.g., {{ }} syntax) in the template.

Requirements:

- Node.js (v18+ recommended) and npm installed.
- Angular CLI installed globally: npm install -g @angular/cli.
- A code editor (e.g., VS Code) with Angular extensions.
- Internet access for initial CLI setup.

Procedure:

1. **Install Angular CLI:**

   Run npm install -g @angular/cli@20.
2. **Create a new project:**
   - Run ng new unit4-angular-lab --inline-style --inline-template --skip-tests --routing=false --style=css --zoneless --ssr=false.
   - Navigate to the project: cd unit4-angular-lab.
3. **Update the component:**
   - Define properties for interpolation.
   - Use inline template for HTML and styles for CSS.
   - Update app.ts.
4. **Run the application:**

   Start with ng serve --open (opens at http://localhost:4200).

Code:

In src\app\app.ts

```
import { Component } from '@angular/core';


@Component({
```

```
  selector: 'app-root',
  template: `
    <div class="hello-container">
      <h1>{{ title }}</h1>
      <p>Hello, {{ userName }}! This is your first Angular component.</p>
      <p>Today's year: {{ currentYear }}</p>
    </div>
  `,
  styles: [
    `
      .hello-container {
        text-align: center;
        padding: 20px;
        background-color: #f0f0f0;
        border-radius: 8px;
        max-width: 600px;
        margin: 20px auto;
      }
      h1 {
        color: #007bff;
      }
      p {
        font-size: 18px;
        color: #333;
      }
    `,
  ],
})
export class App {
  title = 'Welcome to Angular 20!';
  userName = 'Lab Student';
  currentYear = new Date().getFullYear();
}
```

Execution: ng serve --open launches the app at http://localhost:4200.

Output:



Task 2: Implement @if and @for to Dynamically Show/Hide Elements and List Data

Aim:

Create an Angular 20 component that uses @if and @for control-flow directives to dynamically show/hide elements and display a list of data.

Requirements:

- Angular 20 project from Task 1 (angular-lab).
- Angular CLI

Procedure:

1. **Generate a new component:**
    - Run ng generate component user-list.

```
@Maharshi ➜node(22.17.1) git(master) ➜~\OneDrive\Desktop\unit4 ➜ng generate component user-list
CREATE src/app/user-list/user-list.ts (218 bytes)
```

2. **Update the component:**
    - Define a list of users and a toggle flag.
    - Use @if to conditionally show/hide a message.
    - Use @for to iterate over the user list.
    - Apply inline CSS for styling.
3. **Update the app:**
    - Add <app-user-list> to app.ts.
4. **Run and test:**
    - Use ng serve --open and verify dynamic rendering.

Code:

In src\app\user-list\user-list.ts

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [],
  template: `
    <div class="user-list-container">
      <h2>User List</h2>
      <button (click)="toggleDisplay()">Toggle Message</button>
      @if (showMessage) {
      <p class="message">Users are visible below!</p>
      } @for (user of users; track user.id) {
      <div class="user-card">
        <span>{{ user.name }} (ID: {{ user.id }})</span>
      </div>
      } @empty {
      <p>No users available.</p>
      }
    </div>
  `,
  styles: [
    `
      .user-list-container {
        max-width: 500px;
        margin: 20px auto;
        padding: 15px;
        background-color: #e9ecef;
        border-radius: 8px;
      }
      h2 {
        color: #343a40;
        text-align: center;
      }
      button {
```

```
      display: block;
      margin: 10px auto;
      padding: 8px 16px;
      background-color: #28a745;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }
    button:hover {
      background-color: #218838;
    }
    .message {
      color: #007bff;
      font-weight: bold;
      text-align: center;
    }
    .user-card {
      padding: 10px;
      margin: 5px 0;
      background-color: white;
      border: 1px solid #dee2e6;
      border-radius: 4px;
    }
  `,
  ],
})
export class UserListComponent {
  users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Charlie' },
  ];
  showMessage = true;
```

```
  toggleDisplay() {
    this.showMessage = !this.showMessage;
  }
}
```

In src\app\app.ts

```
import { Component } from '@angular/core';
import { UserListComponent } from './user-list/user-list';

@Component({
  selector: 'app-root',
  template: `
    <div class="hello-container">
      <h1>{{ title }}</h1>
      <p>Hello, {{ userName }}! This is your first Angular component.</p>
      <p>Today's year: {{ currentYear }}</p>

      <app-user-list />
    </div>
  `,
  styles: [
    `
      .hello-container {
        text-align: center;
        padding: 20px;
        background-color: #f0f0f0;
        border-radius: 8px;
        max-width: 600px;
        margin: 20px auto;
      }
      h1 {
        color: #007bff;
      }
```

```
      p {
        font-size: 18px;
        color: #333;
      }
    `,
  ],
  imports: [UserListComponent],
})
export class App {
  title = 'Welcome to Angular 20!';
  userName = 'Lab Student';
  currentYear = new Date().getFullYear();
}
```

Output:





Task 3: Build a Login Form with Validation Using a Template-Driven Approach

Aim:

Create an Angular 20 component with a template-driven form for user login, including validation for required fields and email format.

Requirements:

- Angular project from Task 1 (angular-lab).
- Angular CLI (ng g c login-form).
- Dependencies: Angular Forms module.
- A code editor (e.g., VS Code).

Procedure:

1. **Generate a new component:**
   - Run ng generate component login-form.
2. **Implement the form:**
   - Use a template-driven form with ngModel for two-way data binding.
   - Add validation for required fields and email format.
   - Display validation errors and handle form submission.
   - Apply inline CSS for styling.
3. **Update the app:**
   - Add <app-login-form> to app.ts.
4. **Run and test:**
   - Start with ng serve --open and verify form functionality and validation.

Code:

In src\app\login-form\login-form.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-login-form',
  imports: [FormsModule],
  template: `
    <div class="login-container">
      <h2>Login Form</h2>
      <form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
        <div class="form-group">
          <label for="email">Email:</label>
          <input
            type="email"
            id="email"
            name="email"
            [(ngModel)]="user.email"
            required
            email
            #email="ngModel"
            class="form-control"
          />
          @if (email.invalid && (email.dirty || email.touched)) {
          <div class="error">
            @if (email.errors?.['required']) {
            <span>Email is required.</span>
            } @if (email.errors?.['email']) {
            <span>Invalid email format.</span>
            }
          </div>
          }
        </div>
        <div class="form-group">
          <label for="password">Password:</label>
```

```
          <input
            type="password"
            id="password"
            name="password"
            [(ngModel)]="user.password"
            required
            minlength="6"
            #password="ngModel"
            class="form-control"
          />
          @if (password.invalid && (password.dirty || password.touched)) {
          <div class="error">
            @if (password.errors?.['required']) {
            <span>Password is required.</span>
            } @if (password.errors?.['minlength']) {
            <span>Password must be at least 6 characters.</span>
            }
          </div>
          }
        </div>
        <button type="submit" [disabled]="loginForm.invalid" class="submit-
btn">
          Login
        </button>
      </form>
      @if (submitted) {
      <p class="success">Login successful for {{ user.email }}!</p>
      }
    </div>
  `,
  styles: [
    `
      .login-container {
        max-width: 400px;
```

```
    margin: 20px auto;
    padding: 20px;
    background-color: #f8f9fa;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
  }
  h2 {
    text-align: center;
    color: #343a40;
  }
  .form-group {
    margin-bottom: 15px;
  }
  label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
  }
  .form-control {
    width: 100%;
    padding: 8px;
    border: 1px solid #ced4da;
    border-radius: 4px;
  }
  .error {
    color: #dc3545;
    font-size: 14px;
    margin-top: 5px;
  }
  .submit-btn {
    width: 100%;
    padding: 10px;
    background-color: #007bff;
    color: white;
```

```
        border: none;
        border-radius: 4px;
        cursor: pointer;
      }
      .submit-btn:disabled {
        background-color: #6c757d;
        cursor: not-allowed;
      }
      .submit-btn:hover:not(:disabled) {
        background-color: #0056b3;
      }
      .success {
        text-align: center;
        color: #28a745;
        margin-top: 10px;
      }
    `,
  ],
})
export class LoginForm {
  user = { email: '', password: '' };
  submitted = false;

  onSubmit(form: any) {
    if (form.valid) {
      this.submitted = true;
      console.log('Form submitted:', this.user);
    }
  }
}
```

In src\app\app.ts

```
import { Component } from '@angular/core';
import { UserListComponent } from './user-list/user-list';
```

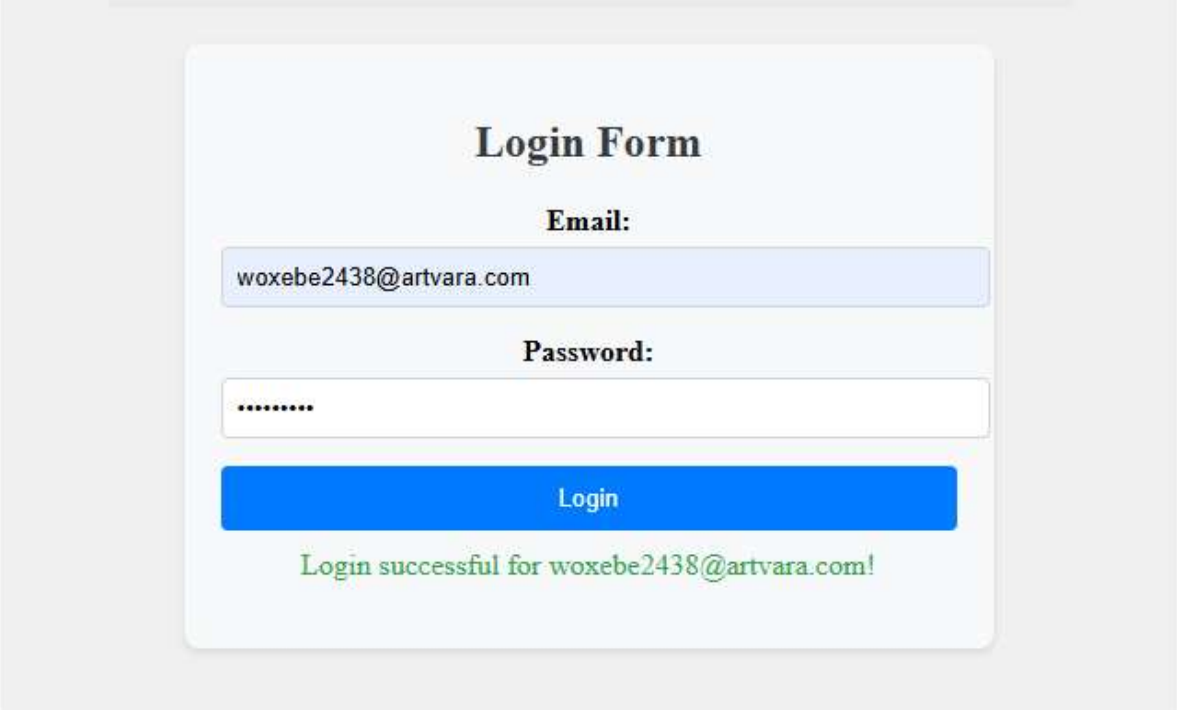
```
import { LoginForm } from './login-form/login-form';

@Component({
  selector: 'app-root',
  template: `
    <div class="hello-container">
      <h1>{{ title }}</h1>
      <p>Hello, {{ userName }}! This is your first Angular component.</p>
      <p>Today's year: {{ currentYear }}</p>

      <app-user-list />
      <app-login-form />
    </div>
  `,
  styles: [
    `
      .hello-container {
        text-align: center;
        padding: 20px;
        background-color: #f0f0f0;
        border-radius: 8px;
        max-width: 600px;
        margin: 20px auto;
      }
      h1 {
        color: #007bff;
      }
      p {
        font-size: 18px;
        color: #333;
      }
    `,
  ],
  imports: [UserListComponent, LoginForm],
```

```
})
export class App {
  title = 'Welcome to Angular 20!';
  userName = 'Lab Student';
  currentYear = new Date().getFullYear();
}
```

Output:



Task 4: Use Built-in Pipes to Transform Data (Uppercase, Lowercase, Date)

Aim:

Create an Angular component that demonstrates the use of built-in pipes (uppercase, lowercase, date) to transform and display data.

Requirements:

- Angular project from Task 1 (angular-lab).
- Angular CLI (ng g c data-transform).
- A code editor (e.g., VS Code).

Procedure:

1. **Generate a new component:**
   - Run ng generate component data-transform.
2. **Implement the component:**
   - Define sample data (e.g., strings and a date).

- Use uppercase, lowercase, and date pipes in the inline template.
- Apply inline CSS for styling.

3. **Update the app:**
   - Add <app-data-transform> to app.component.html.
4. **Run and test:**
   - Start with ng serve --open and verify pipe transformations in the browser.

Code:

In src\app\data-transform\data-transform.ts

```
import { DatePipe, LowerCasePipe, UpperCasePipe } from '@angular/common';
import { Component } from '@angular/core';

@Component({
  selector: 'app-data-transform',
  standalone: true,
  imports: [UpperCasePipe, LowerCasePipe, DatePipe],
  template: `
    <div class="transform-container">
      <h2>Data Transformation with Pipes</h2>
      <div class="data-item">
        <label>Original Name:</label>
        <span>{{ name }}</span>
      </div>
      <div class="data-item">
        <label>Uppercase:</label>
        <span>{{ name | uppercase }}</span>
      </div>
      <div class="data-item">
        <label>Lowercase:</label>
        <span>{{ name | lowercase }}</span>
      </div>
      <div class="data-item">
        <label>Current Date:</label>
        <span>{{ currentDate | date : 'fullDate' }}</span>
      </div>
```

```
    <div class="data-item">
      <label>Short Date:</label>
      <span>{{ currentDate | date : 'short' }}</span>
    </div>
    <div class="data-item">
      <label>Custom Date Format:</label>
      <span>{{ currentDate | date : 'dd MMMM yyyy, h:mm a' }}</span>
    </div>
  </div>
`,
styles: [
  `
    .transform-container {
      max-width: 500px;
      margin: 20px auto;
      padding: 20px;
      background-color: #ffffff;
      border-radius: 8px;
      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
    }
    h2 {
      text-align: center;
      color: #343a40;
    }
    .data-item {
      display: flex;
      justify-content: space-between;
      margin: 10px 0;
      padding: 8px;
      background-color: #f1f3f5;
      border-radius: 4px;
    }
    label {
      font-weight: bold;
```

```
        color: #495057;
      }
      span {
        color: #007bff;
      }
    `,
  ],
})
export class DataTransformComponent {
  name = 'Angular Lab';
  currentDate = new Date();
}
```

In src\app\app.ts

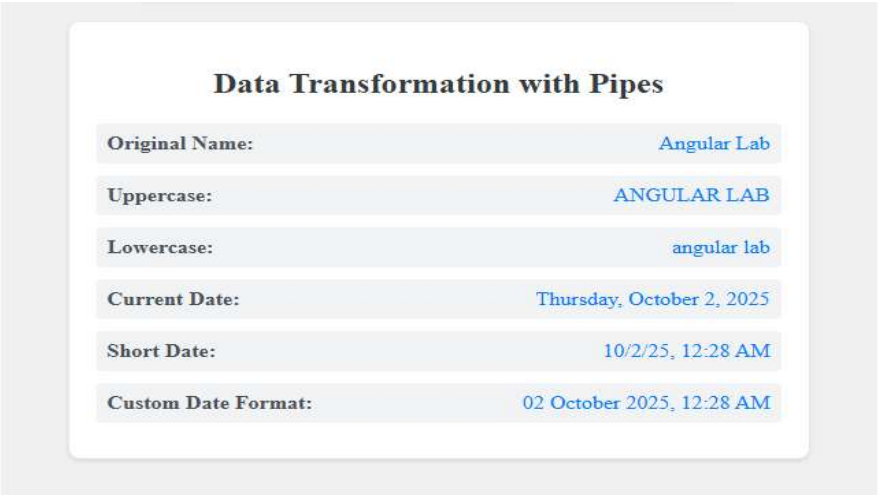
```
import { Component } from '@angular/core';
import { DataTransformComponent } from './data-transform/data-transform';
import { LoginForm } from './login-form/login-form';
import { UserListComponent } from './user-list/user-list';

@Component({
  selector: 'app-root',
  template: `
    <div class="hello-container">
      <h1>{{ title }}</h1>
      <p>Hello, {{ userName }}! This is your first Angular component.</p>
      <p>Today's year: {{ currentYear }}</p>

      <app-user-list />
      <app-login-form />
      <app-data-transform />
    </div>
  `,
  styles: [
    `
```

```
      .hello-container {
        text-align: center;
        padding: 20px;
        background-color: #f0f0f0;
        border-radius: 8px;
        max-width: 600px;
        margin: 20px auto;
      }
      h1 {
        color: #007bff;
      }
      p {
        font-size: 18px;
        color: #333;
      }
    `,
  ],
  imports: [UserListComponent, LoginForm, DataTransformComponent],
})
export class App {
  title = 'Welcome to Angular 20!';
  userName = 'Lab Student';
  currentYear = new Date().getFullYear();
}
```

Output:



UNIT V: Angular Advanced Features & Backend Integration

Services, Dependency Injection, HttpClient, RxJS. Routing, Route Guards, Lazy Loading, Nested Routes,

Tasks:

1. Create an Angular service to fetch and display data using HttpClient.
2. Implement routing between components with RouterLink and route parameters.
3. Add a route guard to restrict access based on login status.
4. Enable lazy loading for a module and observe performance improvements.

Task 1: Create an Angular Service to Fetch and Display Data Using HttpClient

Aim:

Create an Angular service to fetch data from a mock API using HttpClient and display it in a component.

Requirements:

- Angular project.
- Angular CLI installed (`npm install -g @angular/cli`).
- A mock API (e.g., JSONPlaceholder) or a local backend (e.g., Node.js from UNIT II).
- Dependencies: `HttpClientModule` (included in Angular).
- A code editor (e.g., VS Code).

Procedure:

1. **Enable HttpClient:**
   - Import `provideHttpClient` from `@angular/common/http`.
   - Include it in `app.config.ts` providers array.
2. **Generate a service:**
   - Run `ng generate service services/api` to create an API service.
3. **Generate a component:**
   - Run `ng generate component data-fetch`.
4. **Set up the service:**
   - Use `HttpClient` to fetch data from a mock API (e.g., JSONPlaceholder's /users).
   - Handle errors and return data as an observable.
5. **Update the component:**
   - Inject the service and subscribe to the data.
   - Display the data using inline template and styles.
6. **Configure the app:**
   - Import `HttpClientModule` in `app.config.ts`.
   - Add `<app-data-fetch>` to `app.component.html`.

```
import { provideHttpClient } from '@angular/common/http';
import {
  ApplicationConfig,
  provideBrowserGlobalErrorListeners,
  provideZonelessChangeDetection,
} from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZonelessChangeDetection(),
    provideHttpClient(),
  ],
};
```

Generate a API service:

```
@Maharshi ➜ node(22.17.1) git(master) ➜ ~\OneDrive\Desktop\unit4 ➜ ng generate service services/api
CREATE src/app/services/api.ts (115 bytes)
```

Generate a component:

```
@Maharshi ➜ node(22.17.1) git(master) ➜ ~\OneDrive\Desktop\unit4 ➜ ng generate component data-fetch
CREATE src/app/data-fetch/data-fetch.ts (221 bytes)
```

In src\app\services\api.ts

```
import { HttpClient, HttpErrorResponse } from '@angular/common/http';
import { Injectable, signal } from '@angular/core';

export interface User {
  id: number;
  name: string;
  email: string;
```

```
}

@Injectable({
  providedIn: 'root',
})
export class ApiService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/users';

  // signals
  users = signal<User[]>([]);
  errorMessage = signal<string>('');
  loading = signal<boolean>(false);

  constructor(private http: HttpClient) {}

  fetchUsers(): void {
    this.loading.set(true);
    this.errorMessage.set('');
    this.http.get<User[]>(this.apiUrl).subscribe({
      next: (data) => {
        this.users.set(data);
        this.loading.set(false);
      },
      error: (err: HttpErrorResponse) => {
        console.error('API Error:', err.message);
        this.errorMessage.set('Failed to fetch users; please try again
later.');
        this.loading.set(false);
      },
    });
  }
}
```

In src\app\services\api.ts

```
import { CommonModule } from '@angular/common';
import { Component, OnInit } from '@angular/core';
import { ApiService } from '../services/api';

@Component({
  selector: 'app-data-fetch',
  imports: [CommonModule],
  template: `
    <div class="fetch-container">
      <h2>Fetched Users</h2>

      @if (api.errorMessage()) {
      <p class="error">{{ api.errorMessage() }}</p>
      } @else if (api.loading()) {
      <p>Loading users...</p>
      } @else if (api.users().length > 0) {
      <ul>
        @for (user of api.users(); track user.id) {
        <li>{{ user.name }} ({{ user.email }})</li>
        }
      </ul>
      }
    </div>
  `,
  styles: [
    `
      .fetch-container {
        max-width: 600px;
        margin: 20px auto;
        padding: 20px;
        background-color: #f8f9fa;
        border-radius: 8px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
      }
```

```
      h2 {
        text-align: center;
        color: #343a40;
      }
      ul {
        list-style: none;
        padding: 0;
      }
      li {
        padding: 10px;
        margin: 5px 0;
        background-color: white;
        border: 1px solid #dee2e6;
        border-radius: 4px;
      }
      .error {
        color: #dc3545;
        text-align: center;
        font-weight: bold;
      }
    `,
  ],
})
export class DataFetchComponent implements OnInit {
  constructor(public api: ApiService) {}

  ngOnInit(): void {
    this.api.fetchUsers();
  }
}
```

In src\app\app.ts

```
import { Component } from '@angular/core';
import { DataFetchComponent } from './data-fetch/data-fetch';
```

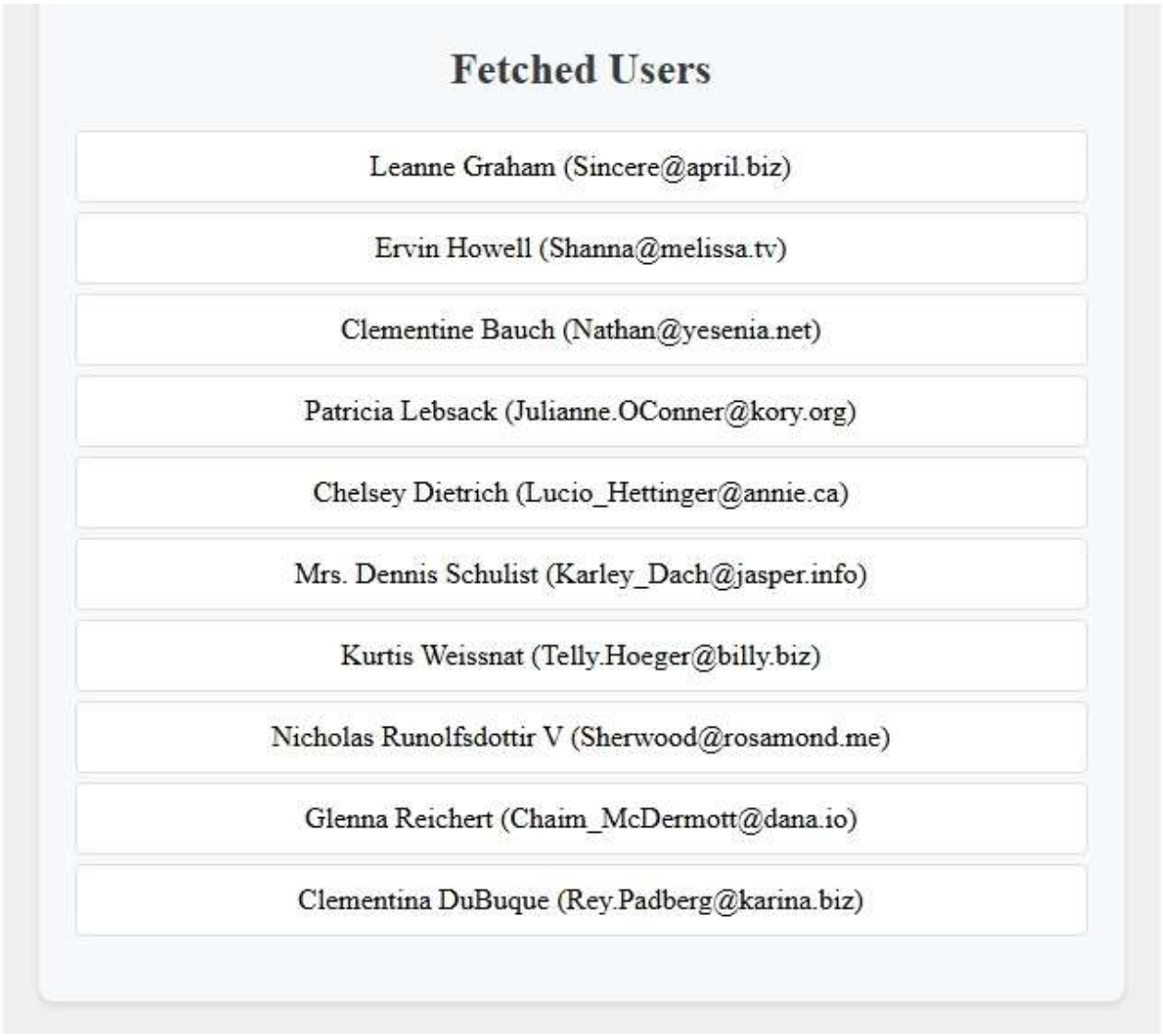
```
import { DataTransformComponent } from './data-transform/data-transform';
import { LoginForm } from './login-form/login-form';
import { UserListComponent } from './user-list/user-list';

@Component({
  selector: 'app-root',
  template: `
    <div class="hello-container">
      <h1>{{ title }}</h1>
      <p>Hello, {{ userName }}! This is your first Angular component.</p>
      <p>Today's year: {{ currentYear }}</p>

      <app-user-list />
      <app-login-form />
      <app-data-transform />
      <app-data-fetch />
    </div>
  `,
  styles: [
    `
      .hello-container {
        text-align: center;
        padding: 20px;
        background-color: #f0f0f0;
        border-radius: 8px;
        max-width: 600px;
        margin: 20px auto;
      }
      h1 {
        color: #007bff;
      }
      p {
        font-size: 18px;
        color: #333;
```

```
      }
    `,
  ],
  imports: [
    UserListComponent,
    LoginForm,
    DataTransformComponent,
    DataFetchComponent,
  ],
})
export class App {
  title = 'Welcome to Angular 20!';
  userName = 'Lab Student';
  currentYear = new Date().getFullYear();
}
```

Output:



Task 2: Implement Routing Between Components with RouterLink and Route Parameters

Aim:

Configure routing in an Angular application to navigate between components using RouterLink and pass route parameters to display dynamic data.

Requirements:

- Angular project (angular-lab from UNIT IV/Task 1).
- Angular CLI (`ng g c user-detail`).
- Dependencies: `Angular Router`.
- A code editor (e.g., VS Code).

Procedure:

1. **Enable routing:**
    - Update the project to include routing (modify `app.config.ts` and create `app.routes.ts`).

2. **Generate a new component:**
    - Run `ng generate component user-detail` for a user detail page.
3. **Define routes:**
    - Set up routes for existing components (e.g., user-list, data-fetch) and the new `user-detail` component with a parameter (`:id`).
4. **Update components:**
    - Add navigation links using `RouterLink` in a component (e.g., user-list).
    - Display route parameter data in user-detail.
5. **Update the app:**
    - Replace `app.ts` content with `<router-outlet>`.
6. **Run and test:**
    - Start with `ng serve --open` and verify navigation and parameter handling.

Code:

Enable Routing in Angular:

In src\app.routes.ts

```
import { Routes } from '@angular/router';
import { DataFetchComponent } from './app/data-fetch/data-fetch';
import { DataTransformComponent } from './app/data-transform/data-transform';
import { LoginFormComponent } from './app/login-form/login-form';
import { UserDetailComponent } from './app/user-detail/user-detail';
import { UserListComponent } from './app/user-list/user-list';

export const routes: Routes = [
  { path: '', component: UserListComponent },
  { path: 'user/:id', component: UserDetailComponent },
  { path: 'login', component: LoginFormComponent },
  { path: 'data-fetch', component: DataFetchComponent },
  { path: 'data-transform', component: DataTransformComponent },
];
```

In src\app\app.config.ts

```
import { provideHttpClient } from '@angular/common/http';
import {
  ApplicationConfig,
  provideBrowserGlobalErrorListeners,
```

```
  provideZonelessChangeDetection,
} from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from '../app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZonelessChangeDetection(),
    provideHttpClient(),
    provideRouter(routes),
  ],
};
```

Generate user-detail component:

```
@Maharshi ➜ node(22.17.1) git(master) ➜ ~\OneDrive\Desktop\unit4 ➜ ng generate component user-detail
CREATE src/app/user-detail/user-detail.ts (224 bytes)
```

Update src\app\user-list\user-list.ts:

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div class="user-list-container">
      <h2>User List</h2>
      <button (click)="toggleDisplay()">Toggle Message</button>
      @if (showMessage) {
      <p class="message">Users are visible below!</p>
      } @for (user of users; track user.id) {
      <div class="user-card">
        <a [routerLink]="['/user', user.id]"
          >{{ user.name }} (ID: {{ user.id }})</a
```

```
      >
    </div>
    } @empty {
    <p>No users available.</p>
    }
  </div>
`,
styles: [
  `
    .user-list-container {
      max-width: 500px;
      margin: 20px auto;
      padding: 15px;
      background-color: #e9ecef;
      border-radius: 8px;
    }
    h2 {
      text-align: center;
      color: #343a40;
    }
    button {
      display: block;
      margin: 10px auto;
      padding: 8px 16px;
      background-color: #28a745;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }
    button:hover {
      background-color: #218838;
    }
    .message {
```

```
        color: #007bff;
        font-weight: bold;
        text-align: center;
      }
      .user-card {
        padding: 10px;
        margin: 5px 0;
        background-color: white;
        border: 1px solid #dee2e6;
        border-radius: 4px;
      }
      .user-card a {
        text-decoration: none;
        color: #007bff;
      }
      .user-card a:hover {
        text-decoration: underline;
      }
    `,
  ],
})
export class UserListComponent {
  users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Charlie' },
  ];
  showMessage = true;

  toggleDisplay() {
    this.showMessage = !this.showMessage;
  }
}
```

In src\app\user-detail\user-detail.ts:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

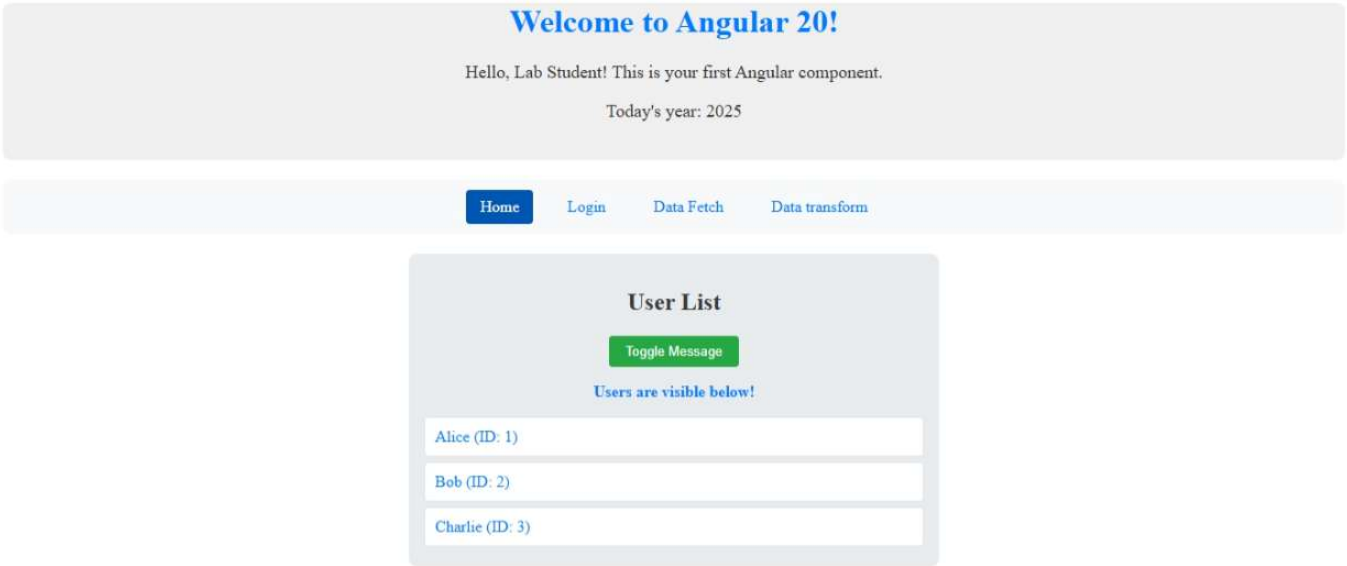
@Component({
  selector: 'app-user-detail',
  standalone: true,
  imports: [],
  template: `
    <div class="detail-container">
      <h2>User Details</h2>
      @if (user) {
      <p>Name: {{ user.name }}</p>
      <p>ID: {{ user.id }}</p>
      } @else {
      <p>User not found.</p>
      }
    </div>
  `,
  styles: [
    `
      .detail-container {
        max-width: 400px;
        margin: 20px auto;
        padding: 20px;
        background-color: #f8f9fa;
        border-radius: 8px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
      }
      h2 {
        text-align: center;
        color: #343a40;
      }
      p {
```

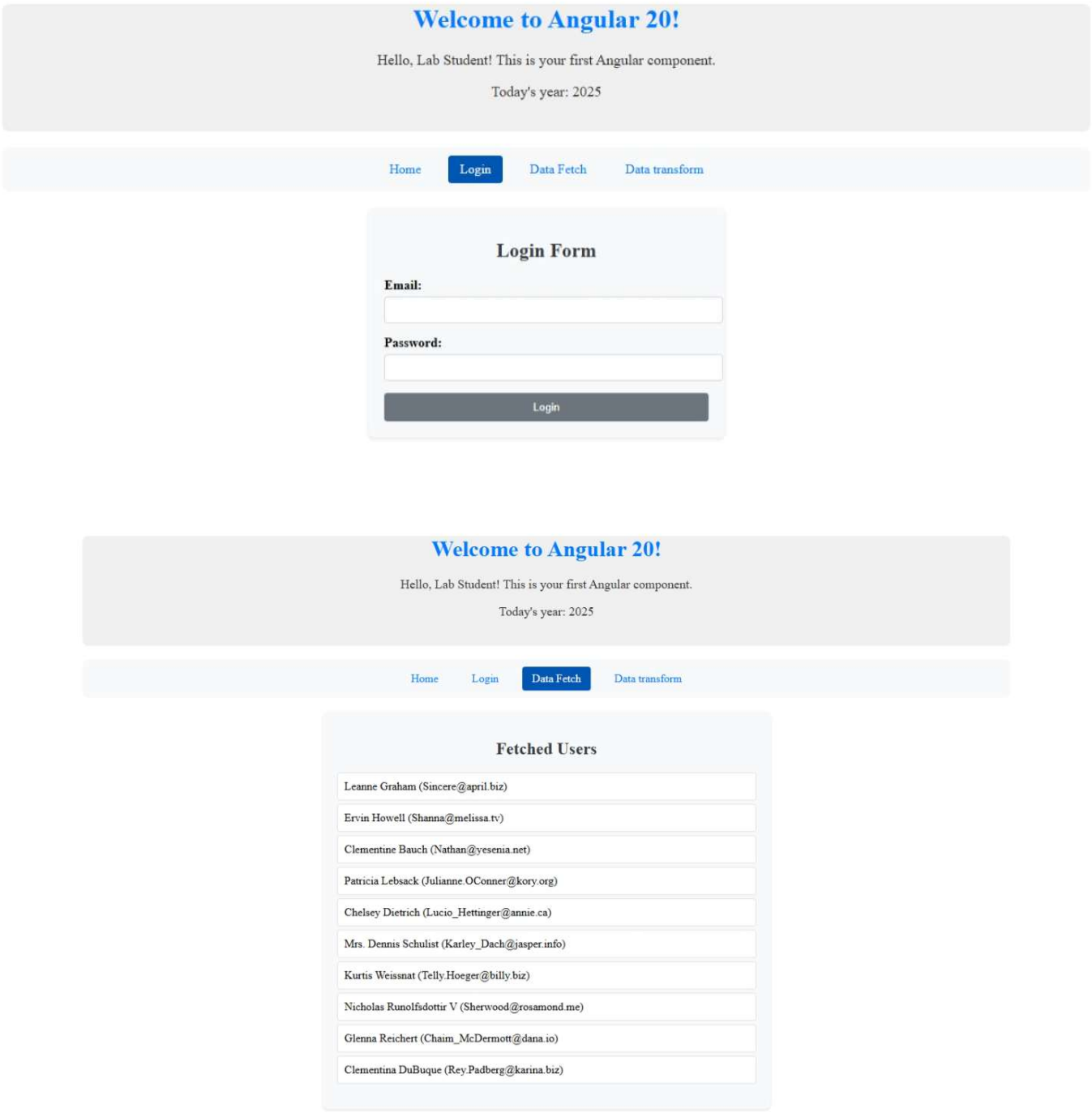
```
        font-size: 18px;
        color: #495057;
      }
    `,
  ],
})
export class UserDetailComponent implements OnInit {
  user: { id: number; name: string } | null = null;

  constructor(private route: ActivatedRoute) {}

  ngOnInit(): void {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    const users = [
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' },
      { id: 3, name: 'Charlie' },
    ];
    this.user = users.find((u) => u.id === id) || null;
  }
}
```

Output:




Welcome to Angular 20!
Hello, Lab Student! This is your first Angular component.
Today's year: 2025
Home
Login
Data Fetch
Data transform
Login Form
Email:
Password:
Login
Welcome to Angular 20!
Hello, Lab Student! This is your first Angular component.
Today's year: 2025
Home
Login
Data Fetch
Data transform
Fetched Users
Leanne Graham (Sincere@april.biz)
Ervin Howell (Shanna@melissa.tv)
Clementine Bauch (Nathan@yesenia.net)
Patricia Lebsack (Julianne.OConner@kory.org)
Chelsey Dietrich (Lucio_Hettinger@annie.ca)
Mrs. Dennis Schulist (Karley_Dach@jasper.info)
Kurtis Weissnat (Telly.Hoeger@billy.biz)
Nicholas Runolfsdottir V (Sherwood@rosamond.me)
Glenna Reichert (Chaim_McDermott@dana.io)
Clementina DuBuque (Rey.Padberg@karina.biz)



Task 3: Add a Route Guard to Restrict Access Based on Login Status

Aim:

Implement an Angular route guard to restrict access to a protected route (e.g., /data) based on login status, using a service to track authentication and inline templates/styles for any updated components.

Requirements:

- Angular 20 project (angular-lab from UNIT IV/Task 1 and UNIT V/Tasks 1–2).
- Angular CLI (`ng g guard auth --implements CanActivate`).
- Dependencies: `Angular Router` and `Forms` (already included).
- Existing login-form component (UNIT IV Task 3) for login functionality.
- A code editor (e.g., VS Code).

Procedure:

1. **Generate a service for authentication:**
   - Run `ng generate service services/auth` to manage login status.
2. **Generate a route guard:**
   - Run `ng generate guard auth --implements CanActivate` to create a guard for route protection.
3. **Update the auth service:**
   - Add methods to handle login/logout and track status.
4. **Implement the guard:**
   - Use the auth service to check login status and redirect unauthorized users to `/login`.
5. **Update routing:**
   - Apply the guard to the `/data` route (protecting `DataFetchComponent`).

6. **Update the login component:**
    - Integrate with the auth service to set login status on successful form submission.
7. **Run and test:**
    - Start with ng serve --open and verify restricted access and redirection.

Code:

Generate a service for authentication:

```
@Maharshi →node(22.17.1) git(master) →~\OneDrive\Desktop\unit4 →ng generate service services/auth
CREATE src/app/services/auth.ts (116 bytes)
```

Generate a route guard:

```
@Maharshi →node(22.17.1) git(master) →~\OneDrive\Desktop\unit4 →ng generate guard auth --implements CanActivate
CREATE src/app/auth-guard.ts (133 bytes)
```

In src\app\services\auth.ts

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class AuthService {
  private isLoggedIn = false;

  login(email: string, password: string): boolean {
    // Simulate authentication (replace with real backend check in
production)
    if (email && password.length >= 6) {
      this.isLoggedIn = true;
      return true;
    }
    return false;
  }

  logout(): void {
    this.isLoggedIn = false;
  }

  getLoginStatus(): boolean {
    return this.isLoggedIn;
```

```
  }
}
```

In src\app\auth-guard.ts

```
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from './services/auth';

@Injectable({
  providedIn: 'root',
})
export class AuthGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.authService.getLoginStatus()) {
      return true;
    }
    this.router.navigate(['/login']);
    return false;
  }
}
```

In src\app.routes.ts

```
import { Routes } from '@angular/router';
import { AuthGuard } from './app/auth-guard';
import { DataFetchComponent } from './app/data-fetch/data-fetch';
import { DataTransformComponent } from './app/data-transform/data-transform';
import { LoginFormComponent } from './app/login-form/login-form';
import { UserDetailComponent } from './app/user-detail/user-detail';
import { UserListComponent } from './app/user-list/user-list';

export const routes: Routes = [
  { path: '', component: UserListComponent },
```

```
  { path: 'user/:id', component: UserDetailComponent },
  { path: 'login', component: LoginFormComponent },
  {
    path: 'data-fetch',
    component: DataFetchComponent,
    canActivate: [AuthGuard],
  },
  { path: 'data-transform', component: DataTransformComponent },
];
```

Update src\app\login-form\login-form.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../services/auth';

@Component({
  selector: 'app-login-form',
  standalone: true,
  imports: [FormsModule],
  template: `
    <div class="login-container">
      <h2>Login Form</h2>
      <form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
        <div class="form-group">
          <label for="email">Email:</label>
          <input
            type="email"
            id="email"
            name="email"
            [(ngModel)]="user.email"
            required
            email
            #email="ngModel"
```

```
        class="form-control"
      />
      @if (email.invalid && (email.dirty || email.touched)) {
      <div class="error">
        @if (email.errors?.['required']) {
        <span>Email is required.</span>
        } @if (email.errors?.['email']) {
        <span>Invalid email format.</span>
        }
      </div>
      }
    </div>
    <div class="form-group">
      <label for="password">Password:</label>
      <input
        type="password"
        id="password"
        name="password"
        [(ngModel)]="user.password"
        required
        minlength="6"
        #password="ngModel"
        class="form-control"
      />
      @if (password.invalid && (password.dirty || password.touched)) {
      <div class="error">
        @if (password.errors?.['required']) {
        <span>Password is required.</span>
        } @if (password.errors?.['minlength']) {
        <span>Password must be at least 6 characters.</span>
        }
      </div>
      }
    </div>
```

```
        <button type="submit" [disabled]="loginForm.invalid" class="submit-
btn">
          Login
        </button>
      </form>
      @if (submitted) {
      <p class="success">Login successful for {{ user.email }}!</p>
      }
    </div>
  `,
  styles: [
    `
      .login-container {
        max-width: 400px;
        margin: 20px auto;
        padding: 20px;
        background-color: #f8f9fa;
        border-radius: 8px;
        box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
      }
      h2 {
        text-align: center;
        color: #343a40;
      }
      .form-group {
        margin-bottom: 15px;
      }
      label {
        display: block;
        margin-bottom: 5px;
        font-weight: bold;
      }
      .form-control {
        width: 100%;
```

```
      padding: 8px;
      border: 1px solid #ced4da;
      border-radius: 4px;
    }
    .error {
      color: #dc3545;
      font-size: 14px;
      margin-top: 5px;
    }
    .submit-btn {
      width: 100%;
      padding: 10px;
      background-color: #007bff;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }
    .submit-btn:disabled {
      background-color: #6c757d;
      cursor: not-allowed;
    }
    .submit-btn:hover:not(:disabled) {
      background-color: #0056b3;
    }
    .success {
      text-align: center;
      color: #28a745;
      margin-top: 10px;
    }
  `,
 ],
})
export class LoginFormComponent {
```

```
  user = { email: '', password: '' };
  submitted = false;

  constructor(private authService: AuthService, private router: Router) {}

  onSubmit(form: any) {
    if (form.valid) {
      const success = this.authService.login(
        this.user.email,
        this.user.password
      );
      if (success) {
        this.submitted = true;
        this.router.navigate(['/data-fetch']);
      } else {
        this.submitted = false;
      }
    }
  }
}
```

Update src\app\app.ts

```
import { Component } from '@angular/core';
import {
  Router,
  RouterLink,
  RouterLinkActive,
  RouterOutlet,
} from '@angular/router';
import { AuthService } from './services/auth';

@Component({
  selector: 'app-root',
  standalone: true,
```

```
imports: [RouterOutlet, RouterLink, RouterLinkActive],

template: `
  <div>
    <div class="hello-container">
      <h1>{{ title }}</h1>
      <p>Hello, {{ userName }}! This is your first Angular component.</p>
      <p>Today's year: {{ currentYear }}</p>
    </div>
    <nav class="nav">
      <a
        routerLink="/"
        routerLinkActive="active"
        [routerLinkActiveOptions]="{ exact: true }"
        >Home</a
      >
      <a
        routerLink="/data-fetch"
        routerLinkActive="active"
        [routerLinkActiveOptions]="{ exact: true }"
        >Data Fetch</a
      >
      <a
        routerLink="/data-transform"
        routerLinkActive="active"
        [routerLinkActiveOptions]="{ exact: true }"
        >Data transform</a
      >

      @if (authService.getLoginStatus()) {
      <button (click)="logout()">Logout</button>
      } @else {
      <a
        routerLink="/login"
```

```
        routerLinkActive="active"
        [routerLinkActiveOptions]="{ exact: true }"
        >Login</a
      >
      }
    </nav>
    <router-outlet></router-outlet>
  </div>
`,
styles: [
  `
    .hello-container {
      text-align: center;
      padding-bottom: 20px;
      background-color: #f0f0f0;
      border-radius: 8px;
      width: 100%;
      margin: 20px auto;
    }
    h1 {
      color: #007bff;
    }
    p {
      font-size: 18px;
      color: #333;
    }
    .nav {
      display: flex;
      justify-content: center;
      gap: 20px;
      margin: 20px 0;
      padding: 10px 0;
      background-color: #f8f9fa;
      border-radius: 8px;
```

```
      }

      .nav a {
        text-decoration: none;
        color: #007bff;
        font-weight: 500;
        padding: 8px 14px;
        border-radius: 4px;
        transition: background-color 0.2s, color 0.2s;
      }

      .nav a:hover {
        background-color: #007bff;
        color: #fff;
      }

      .nav a.active {
        background-color: #0056b3;
        color: #fff;
      }
    `,
  ],
})
export class App {
  title = 'Welcome to Angular 20!';
  userName = 'Lab Student';
  currentYear = new Date().getFullYear();

  constructor(public authService: AuthService, private router: Router) {}

  logout() {
    this.authService.logout();
    this.router.navigate(['/login']);
  }
```

```
}
```

Task 4: Enable Lazy Loading for a Module and Observe Performance Improvements

Aim:

Configure lazy loading for a feature module containing the DataFetchComponent in an Angular application to improve initial load performance.

Requirements:

- Angular project (angular-lab from UNIT IV and UNIT V/Tasks 1–3).
- Angular CLI (ng g module data --route data --module app).
- Dependencies: Angular Router and HttpClient (already included).
- A code editor (e.g., VS Code).
- Browser DevTools for performance analysis.

Procedure:

1. **Generate a feature module:**
   - Run ng generate module data --route data --module app to create a DataModule with routing and lazy load it in the main app.
   - This generates app.module.ts in src/data folder.
2. **Move DataFetchComponent:**
   - Move data-fetch component files to the data module folder and update imports.
3. **Update the component:**
   - Ensure DataFetchComponent uses inline template and styles (already done in Task 1).
4. **Configure lazy loading:**
   - Update app.routes.ts to lazy load DataModule.
   - Adjust app.ts to maintain navigation.
5. **Run and test:**
   - Start with ng serve --open and use DevTools to observe performance improvements.

Code:

In src\data\app.module.ts

```
import { CommonModule } from '@angular/common';
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { DataFetchComponent } from '../app/data-fetch/data-fetch';

const routes: Routes = [{ path: '', component: DataFetchComponent }];
```

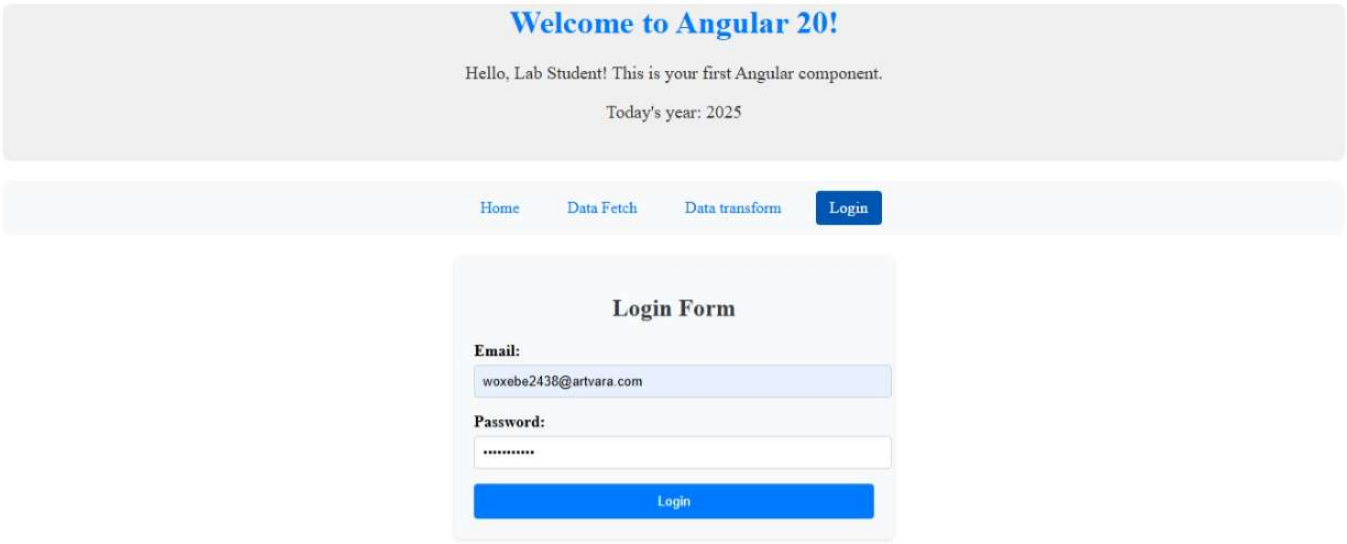
```
@NgModule({
  declarations: [],
  imports: [CommonModule, RouterModule.forChild(routes)],
})
export class DataModule {}
```

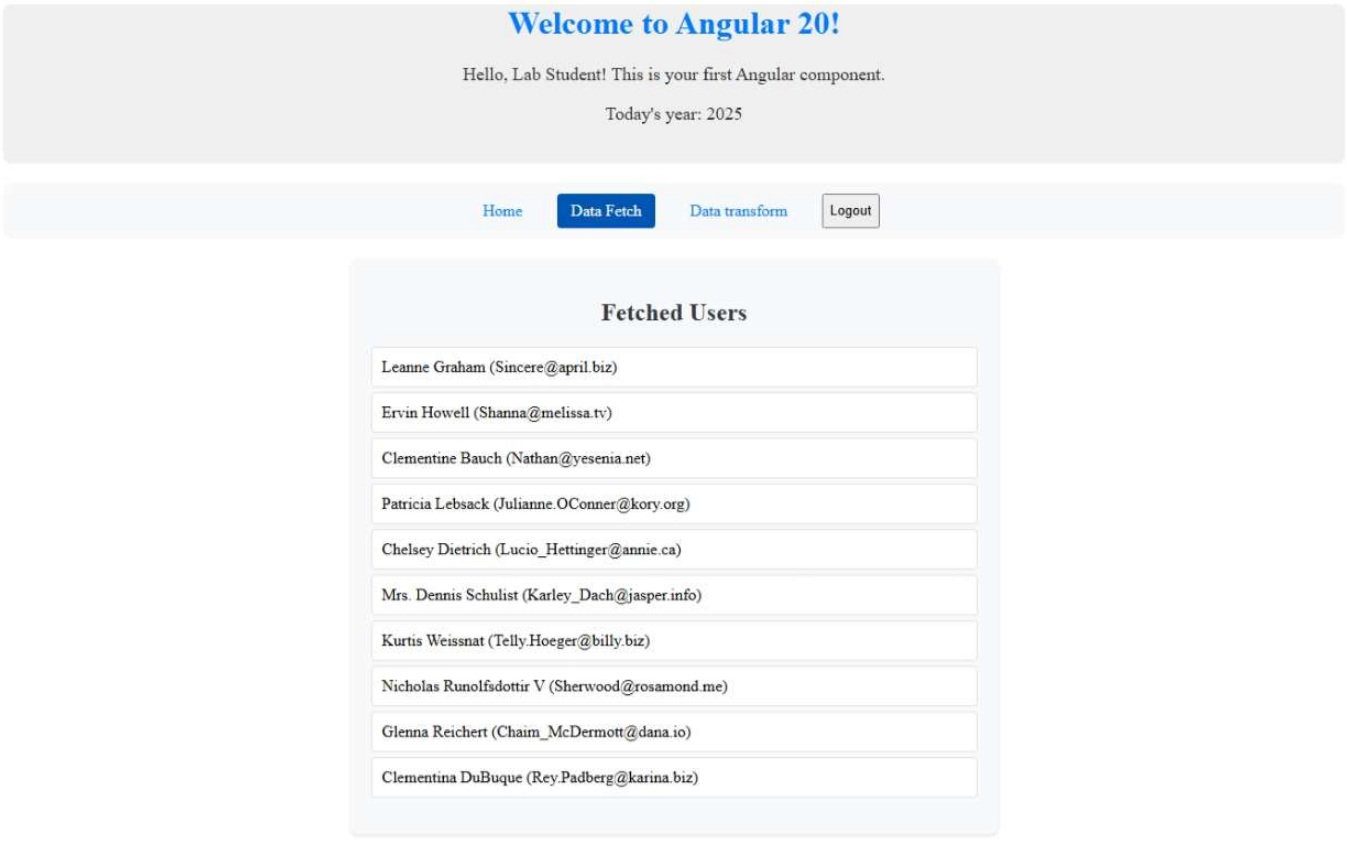
Update src\app.routes.ts

```
import { Routes } from '@angular/router';
import { AuthGuard } from './app/auth-guard';
import { DataTransformComponent } from './app/data-transform/data-transform';
import { LoginFormComponent } from './app/login-form/login-form';
import { UserDetailComponent } from './app/user-detail/user-detail';
import { UserListComponent } from './app/user-list/user-list';

export const routes: Routes = [
  { path: '', component: UserListComponent },
  { path: 'user/:id', component: UserDetailComponent },
  { path: 'login', component: LoginFormComponent },
  {
    path: 'data-fetch',
    loadChildren: () => import('./data/app.module').then((m) =>
m.DataModule),
    canActivate: [AuthGuard],
  },
  { path: 'data-transform', component: DataTransformComponent },
];
```

Output:


Welcome to Angular 20!
Hello, Lab Student! This is your first Angular component.
Today's year: 2025
Home
Data Fetch
Data transform
Login
Login Form
Email:
woxebe2438@artvara.com
Password:
Login


Welcome to Angular 20!
Hello, Lab Student! This is your first Angular component.
Today's year: 2025
Home
Data Fetch
Data transform
Logout
Fetched Users
Leanne Graham (Sincere@april.biz)
Ervin Howell (Shanna@melissa.tv)
Clementine Bauch (Nathan@yesenia.net)
Patricia Lebsack (Julianne.OConner@kory.org)
Chelsey Dietrich (Lucio_Hettinger@annie.ca)
Mrs. Dennis Schulist (Karley_Dach@jasper.info)
Kurtis Weissnat (Telly.Hoeger@billy.biz)
Nicholas Runolfsdottir V (Sherwood@rosamond.me)
Glenna Reichert (Chaim_McDermott@dana.io)
Clementina DuBuque (Rey.Padberg@karina.biz)